arXiv:2407.00144v2 [cs.RO] 4 Jun 2025

SCOPE: Stochastic Cartographic Occupancy Prediction Engine for Uncertainty-Aware Dynamic Navigation

Zhanteng Xie, *Member, IEEE*, and Philip Dames, *Member, IEEE*

***Abstract*—This article presents a family of Stochastic Cartographic Occupancy Prediction Engines (SCOPEs) that enable mobile robots to predict the future states of complex dynamic environments. They do this by accounting for the motion of the robot itself, the motion of dynamic objects, and the geometry of static objects in the scene, and they generate a range of possible future states of the environment. These prediction engines are software-optimized for real-time performance for navigation in crowded dynamic scenes, achieving up to 89 times faster inference speed and 8 times less memory usage than other state-of-the-art engines. Three simulated and real-world datasets collected by different robot models are used to demonstrate that these proposed prediction algorithms are able to achieve more accurate and robust stochastic prediction performance than other algorithms. Furthermore, a series of simulation and hardware navigation experiments demonstrate that the proposed predictive uncertainty-aware navigation framework with these stochastic prediction engines is able to improve the safe navigation performance of current state-of-the-art model- and learning-based control policies.**

***Index Terms*—Deep Learning in Robotics and Automation, Reactive and Sensor-Based Planning, Learning and Adaptive Systems, Environment Prediction.**

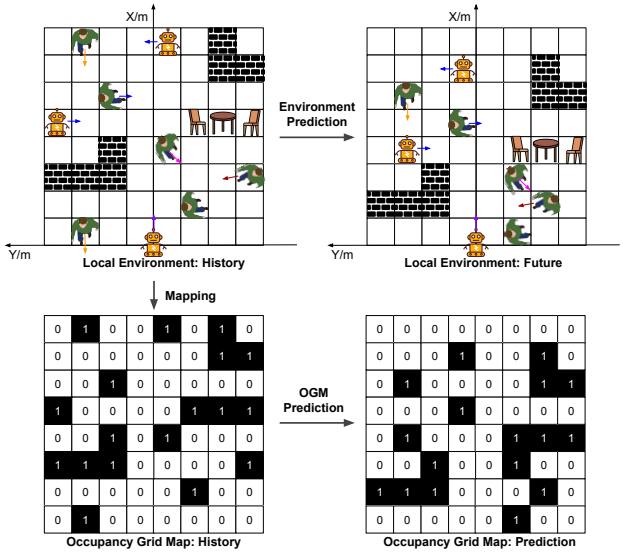


Fig. 1. A simple illustration of the occupancy grid map prediction problem. In a complex dynamic environment with many pedestrians, robots, tables, chairs and walls, colored arrows indicate the velocity of each agent.

I. Introduction

AUTONOMOUS mobile robots are beginning to enter people's lives and are trying to help us provide different last mile delivery services, such as moving goods in warehouses or hospitals and assisting grocery shoppers [1]–[3]. To realize this vision, mobile robots are required to safely and efficiently navigate through complex and dynamic environments filled not only with static obstacles (*e.g.,* tables, chairs, and walls) but also with many moving people and/or other mobile robots. The first prerequisite for robots to navigate and perform tasks is to use their sensors to perceive the surrounding environment. This work focuses on the next step, which is to accurately and reliably predict how the surrounding environment will change based on these sensor data, as shown in Fig. 1. This will allow a robot to proactively act based on its predictions and the associated uncertainty to avoid potential future collisions, a key part of improving autonomous robot navigation. Note that since this general perception-prediction-control navigation framework is a complex and resource-intensive system, it is very important to make these algorithms hardware-friendly (*e.g.,* using smaller computational power, memory usage, and storage usage) and run in real-time, especially for mobile robots with limited resources. A well-performing predictor is useless for practical robotics applications if it consumes a lot of memory and/or cannot run in real-time on a resource-limited robot.

In this article, we propose a family of deep neural network (DNN)-based Stochastic Cartographic Occupancy Prediction Engines (*i.e.,* SCOPE, SCOPE++, and SO-SCOPE) for resource-constrained mobile robots to provide stochastic future state predictions and enable uncertainty-aware navigation in crowded dynamic scenes. This article is an evolution of our previous work [4], in which we defined the architecture of the SCOPE and SCOPE++,[1] validated their ability to predict occupancy grid maps (OGMs), and demonstrated their utility for robotic navigation. The primary extensions to [4] include: 1) providing comprehensive OGM prediction performance and resource usage evaluations that include three new state-of-the-art algorithms, 2) mathematically modeling the output characteristics of the prior work to understand its performance, particularly around the statistical distribution of the VAE, 3) designing a novel software-optimized SO-SCOPE

*This work was funded by Temple University.

Zhanteng Xie and Philip Dames are with the Department of Mechanical Engineering, Temple University, Philadelphia, PA, USA {zhanteng.xie, pdames}@temple.edu

Multimedia are available: https://youtu.be/8TtHTtJzuc8

[1]In our previous work [4] we used the acronym SOGMP (Stochastic Occupancy Grid Map Predictor) instead of SCOPE.

predictor that significantly improves the inference speed and addresses the memory-intensive nature of the previous sampling-based SCOPE and SCOPE++ predictors, 4) extending the previous predictive uncertainty-aware navigation framework to enable it to be integrated with resource-intensive learning-based control policies, and 5) providing additional simulation experiments and real-world experiments to demonstrate the performance of these algorithms.

Therefore, integrating our previous work [4] and providing more technically improved materials and more complete experiments, this article presents six primary contributions:

1) We design an algorithmic pipeline called SCOPE++ that can use a short history of robot odometry and lidar measurements to predict a distribution of potential future robot/environment states. SCOPE++ includes modules to compensate for the ego-motion of the robot, to segment static/dynamic objects in the scene, to predict future scenes using a ConvLSTM network, and to sample other future scenes using a variational autoencoder (VAE).
2) We analyze the running time and memory usage of each module of SCOPE++ to identify computational bottlenecks. Based on this, we compress the VAE by performing an in-depth statistical analysis of its output and by using knowledge distillation techniques. The resulting software-optimized SCOPE (SO-SCOPE) achieves slightly better performance while consuming less memory, performing faster inference, and running in real-time with other resource-intensive algorithms on resource-constrained mobile robot hardware.
3) We validate the ability of our SCOPE predictors (*i.e.,* SCOPE++, SCOPE, and SO-SCOPE) to predict OGMs using three OGM datasets (each of which comes from a different robot model) and provide a comprehensive benchmark of prediction performance and resource usage using six state-of-the-art algorithms. We find that the SCOPE family achieves smaller absolute errors, higher structural similarity, higher tracking accuracy, and lower computational resource requirements than other state-of-the-art methods (*i.e.,* ConvLSTM [5], DeepTracking [6], PhyDNet [7], SAAConvLSTM [8], TAAConvLSTM [8], and LOPR [9]). We also perform a detailed analysis of the correctness, diversity, and consistency of the uncertainty estimates from the SCOPE family.
4) We propose a costmap-based predictive uncertainty-aware navigation framework to incorporate OGM prediction and its uncertainty information into current existing navigation control policies to improve their safe navigation performance in crowded dynamic scenes.
5) We validate the navigation performance in simulated 3D environments with varying crowd densities and real-world experiments. We find that the predictive uncertainty-aware navigation framework combined with our proposed SCOPE family can improve the navigation performance and safety of extant control policies relative to state-of-the-art solutions, including a model-based controller [10], a supervised learning-based approach [11], and two deep reinforcement learning (DRL)-based approaches [12], [13].
6) We open source the OGM prediction code with the OGM dataset [14] (https://github.com/TempleRAIL/scope) and its predictive uncertainty-aware navigation framework (https://github.com/TempleRAIL/scope_nav).

II. Related Works

In this section, we provide a detailed description of prior work on environment prediction, deep neural network compression, and uncertainty-aware navigation.

A. Environment Prediction

Environment prediction remains an open problem as the future state of the environment is unknown, complex, and stochastic. Many interesting works have focused on this prediction problem. Traditional object detection and tracking methods [15], [16] use multi-stage procedures, hand-designed features, and explicitly detect and track objects. More recently, deep learning (DL)-based methods that are detection and tracking-free have been able to obtain more accurate predictions [6], [8], [17]–[20]. Occupancy grid maps (OGMs) are the most common environment representation in these methods. This transforms the complex environment prediction problem into an OGM prediction problem, outlined in Fig. 1. Since OGMs can be treated as images (both are 2D arrays of data), the multi-step OGM prediction problem can be thought of as a video prediction task, a well-studied problem in machine learning.

The most common technique for OGM prediction uses recurrent neural networks (RNNs), which are widely used in video prediction (*e.g.,* ConvLSTM [5], PredNet [21], and PhyDNet [7]). For example, Ondruska et al. [6] first propose an RNN-based deep tracking framework to directly track and predict unoccluded OGM states from raw sensor data. Itkina et al. [17] directly adapt PredNet [21] to predict the dynamic OGMs (DOGMas) in urban scenes. Following this line of thought, Toyungyernsub et al. [18] decouple the static and dynamic OGMs and propose a double-prong PredNet to predict occupancy states of the environment. Similarly, Schreiber et al. [19], [20] embed the ConvLSTM units in the U-Net architecture to capture spatio-temporal information of DOGMs and predict them in the stationary vehicle setting. Lange et al. [8] propose two attention-augmented ConvLSTM networks to capture long-range dependencies and predict future OGMs in the moving vehicle setting. However, these image-based works only focus on improving network architectures and just treat the OGMs as images, assuming their network architectures can implicitly capture useful information from the kinematics and dynamics behind the environment with sufficient good data.

While pure image-based methods are simple and explicitly ignore the dynamic information of the environment, many other DL-based approaches exploit the ego-motion information and motion flow of the environment to improve the OGM prediction accuracy. By using a combination of input placement and recurrent states shifting to compensate for the ego-motion, Schreiber et al. [22] extend their previous image-based works [19], [20] to predict DOGMs in moving ego-vehicle scenarios. Dequaire et al. [23] extend the deep tracking framework [6] and propose a gated recurrent unit (GRU)-based

deep tracking network with a spatial transformer module (STM) for ego-motion compensation to predict multi-steps future states in stationary and moving vehicle settings. Song et al. [24] propose a GRU-based LiDAR-FlowNet to estimate the forward and backward motion flow between two consecutive OGMs and predict future OGMs. Thomas et al. [25], [26] directly encode spatiotemporal information into the world coordinate frame and propose a 3D-2D feedforward architecture, called DeepSOGM, to predict futures. By considering the ego-motion and motion flow together, Mohajerin et al. [27] first use the standard geometric image transformation to compensate for the ego-motion of the vehicle, then propose a ConvLSTM-based difference learning architecture to extract the motion difference between consecutive OGMs, and finally predict multi-step future states. However, most of these motion-based works are designed for vehicle scenarios, and their network models are memory-intensive and computationally intensive, putting them out of the range of resource-limited mobile robots. Furthermore, all the above-described works only provide deterministic OGM predictions and cannot estimate the uncertainty of future states, which is a key point in helping robots operate in dynamic hazardous environments. To address this issue, we propose VAE-based stochastic OGM predictors for resource-constrained robots, namely SCOPE++ and SCOPE, both of which predict a distribution of possible future states of dynamic scenes.

B. Deep Neural Network Optimization

Due to limited computational resources, mobile robots always need hardware-friendly deep learning algorithms. Many researchers have been working on accelerating deep learning algorithms from hardware and software aspects, or both. Compared to hardware acceleration, which requires significant redesign and manufacturing changes using specialized hardware architectures such as field programmable gate arrays (FPGAs), neural processing units (NPUs), and custom application-specific integrated circuits (ASICs) (most robots are not equipped with these specialized hardware computing devices), software acceleration offers greater advantages by enabling flexibility and adaptability through modifications to deep neural networks and other software optimizations.

As summarized in surveys [28], [29], the main methods of software acceleration can be divided into three categories: 1) network pruning, 2) weight quantization and sharing, and 3) knowledge distillation. The key idea of network pruning is to analyze the different components of complex DNNs and remove unimportant components, such as unimportant channels [30], [31], unimportant kernel filters [32], [33], unimportant network connections [34], [35], and even unimportant layers [36], [37]. However, network pruning requires expertise and experience to analyze the network model and can easily lead to performance degradation. To avoid the extra knowledge of pruning, many researchers focus on network model weight quantization and sharing, such as using Huffman coding to quantize weights [38], reduce the number of bits representing model weights [39], [40], and share weights between different network connections or different layers [41], [42]. However, weight quantization often requires a good trade-off between performance and bit quantization, and requires computing devices to support its low-precision arithmetic operations. To keep network model performance and compress neural network, Hinton et al. [43] first proposed the knowledge distillation technique, the key idea of which is to train a small and simple "student" network to learn the output of a large and complex "teacher" network. Following the basic distillation idea, many improvement variants are proposed to reduce the learning gap between the "student" network and the "teacher" network such as adding a middle-size network as the "teaching assistant" [44], [45], using early-stopped knowledge distillation "teacher" network [46], and incorporating cross-domain distillation training [47]. Although knowledge distillation requires the selection of appropriate "student" or "teacher" networks, it provides more freedom to choose different compressed network architectures and provides the potential possibility of obtaining compressed "student" networks with better generalization capabilities. Therefore, we chose to combine the knowledge distillation technique with the uncertainty quantification module to software-optimize our proposed VAE-based stochastic OGM predictors and make them hardware-friendly for resource-constrained robots.

C. Uncertainty-Aware Navigation

As we previously discussed, while most approaches [10]–[13] use the past or current environment states (*i.e.,* raw sensor data and preprocessed data representations) as perception information for their navigation control policies, many methods [48]–[53] believe that predicted future states can help robots better avoid collisions with pedestrians, and start utilizing the predicted environment states (*e.g.,* pedestrian trajectory prediction and latent state prediction) to improve navigation performance through pedestrian crowds. However, most prediction-based navigation policies assume the outcomes of the interactive environments and their control actions are deterministic. Due to the stochastic nature of the robot and its surrounding environments, there are many uncertainties in environment prediction and control actions. To leverage the uncertainty information in the environmental future states and provide robust and reliable navigation behavior, Kahn et al. [54] propose a collision prediction network to estimate the future collision uncertainty and incorporate the predicted collision uncertainty into Model Predictive Control (MPC) framework to enable uncertainty-aware navigation behavior. Similarly, under the MPC framework, Lütjens et al. [55] use an ensemble of LSTM networks with dropout and bootstrapping to estimate prediction collision probabilities and provide uncertainty-aware navigation around pedestrians. Tang et al. [56] use the uncertainty-aware potential field to process prediction uncertainty and incorporate it into the MPC framework to provide predictive uncertainty-aware navigation. Furthermore, Sekiguchi et al. [57] convert the uncertainty information of predicted human trajectories into reliability values and incorporate it into a nonlinear MPC framework to realize safe human-followings. However, all these uncertainty-aware works focus on applying their prediction uncertainty information into the MPC framework and cannot be directly adapted to the currently existing model-based and learning-based control policies.

To easily incorporate prediction uncertainty information into existing model-based and learning-based control policies, some works convert prediction uncertainty information as obstacle costmaps to improve existing path planning modules. For example, Georgakis et al. [58] propose a UPEN uncertainty-driven planner by combining OGM prediction uncertainty costmaps with an RRT planner [59] and a DD-PPO control policy [60]. However, this UPEN planner only works in static environments. To enable dynamic navigation, Thomas et al. [26] add OGM prediction costmaps into the timed-elastic-band (TEB) planner of the ROS navigation stack framework. However, their proposed DeepSOGM predictor is a time-consuming 3D network and only contains prediction information but cannot provide and utilize prediction uncertainty information for dynamic navigation. To address this issue, we follow the costmap-based navigation framework and propose a hardware-friendly predictive uncertainty-aware navigation framework to incorporate the prediction and uncertainty information into existing model-based and learning-based control policies to improve their safe navigation performance.

III. Stochastic Cartographic Occupancy Prediction Engine

This section begins by formulating the problem of OGM prediction in dynamic scenes. It will then go on to describe our VAE-based predictors (SCOPE series), describing the system architecture as well as how to handle robot motion, pedestrian motion, and static obstacles in dynamic environments.

A. Problem Formulation

We consider the problem of a mobile robot moving through an environment filled with both static objects (*e.g.,* walls) and dynamic objects (*e.g.,* people). As the robot moves, we assume that it can obtain accurate estimates of its relative pose and velocity from odometry sensors or other localization algorithms in a short period of time (on the order of 1 s). We denote the pose and velocity of the robot at time t by $\mathbf{x}_t = [x_t \ y_t \ \theta_t]^T$ and $\mathbf{u}_t = [v_t \ w_t]^T$ respectively. We assume the robot is equipped with a 2D lidar sensor. Let $\mathbf{y}_t = [\mathbf{r}_t \ \mathbf{b}_t]^T$ denote the lidar measurements (range $\mathbf{r}$ and bearing $\mathbf{b}$) at time t. We assume the world is 2.5D, as is commonly done in mobile robotics applications. Thus, we can represent the environment using a 2D occupancy grid map (OGM). Let $\mathbf{o}_t$ denote the OGM at time t. We will consider OGMs that are both binary (*i.e.,* each cell is either occupied or free) and probabilistic (*i.e.,* each cell has a probability of being occupied).

Given a history of τ lidar measurements $\mathbf{y}_{t-\tau:t}$ and robot states $\{\mathbf{x}_{t-\tau:t}, \mathbf{u}_{t-\tau:t}\}$, we consider the problem of predicting the distribution of future states (*i.e.,* OGMs) of the environment $\mathbf{o}_{t+1}$. For notational compactness, let $\mathbf{d}_{t-\tau:t} = \{\mathbf{x}_{t-\tau:t}, \mathbf{u}_{t-\tau:t}, \mathbf{y}_{t-\tau:t}\}$ denote the history of data over the given window. We can formulate this as a prediction model:

$$p_\theta(\mathbf{o}_{t+1} \mid \mathbf{d}_{t-\tau:t}), \tag{1}$$

where θ are the model parameters, and the goal is to find the optimal θ^* to maximize (1). Note that this model can also be used in a rollout to predict multiple time steps into the future, *i.e.,* using $t+1$ to predict $t+2$, etc.

Note that we set the lidar history to $\tau = 10$, the sampling rate is 10 Hz, and limit the physical size of OGMs to $[0, 6.4]$ m along the x-axis (forward) and $[-3.2, 3.2]$ m along the y-axis (left) in the robot's local coordinate frame. We use a cell size of 0.1 m, resulting in 64×64 OGMs. These settings are consistent with other works on mobile robot navigation [61]. All data $\mathbf{o}, \mathbf{u}, \mathbf{y}$ are represented in the robot's *local* coordinate frame.

B. Image-Based Prediction

An OGM can be considered a grayscale image, with the probability of occupancy defining the "color" in each cell of a regular grid. Using this interpretation, image/video prediction algorithms could be used to generate the next OGM given a short sequence of past OGMs [17], [19], [20]. The prediction model (1) of image-based approaches is rewritten as

$$\mathbf{o}^*_{t+1} = \arg\max \ p_\theta(\mathbf{o}_{t+1} \mid \mathbf{d}_{t-\tau:t}) = \ f_\theta(\mathbf{o}_{t-\tau:t}), \tag{2a}$$

$$\mathbf{o}_{t-\tau:t} = \ \psi(\mathbf{y}_{t-\tau:t}), \tag{2b}$$

where $f_\theta(\cdot)$ is a DNN model, and $\psi(\cdot)$ is the conversion function to convert the lidar measurements to the binary OGMs in the robot's local frame (*i.e.,* using ray tracing). From this image-based model (2), we can see that image-based methods explicitly ignore the kinematics and dynamics of the robot and surrounding objects (*i.e.,* they assume motion can be implicitly captured by powerful network architectures and enough good data), and fail to provide a range of possible and reliable OGM predictions (*i.e.,* they assume a deterministic future).

C. SCOPE++

Based on the limitations of image-based methods above, we argue that: 1) the future state of the environment explicitly depends on the motion of the robot itself, the motion of dynamic objects, and the state of static objects within the environment; 2) the future state of the environment is stochastic and unknown, and a range of possible future states helps provide robust predictions. With these two assumptions, we fully and explicitly exploit the kinematic and dynamic information of these three different types of objects, utilize the VAE-based network to provide stochastic predictions, and finally propose our novel stochastic OGM predictors SCOPE series (shown in Fig. 2) to predict the future state of the environment. We first create a variant called SCOPE++ that explicitly separates out static objects. Since SCOPE++ is a superset of SCOPE, the rest of this section will focus on SCOPE++. The prediction model (1) of our SCOPE++, outlined in Fig. 2, can be rewritten as

$$\mathbf{o}_{t+1} \sim p_\theta(\mathbf{o}_{t+1} \mid \mathbf{d}_{t-\tau:t}) = p_\theta(\mathbf{o}_{t+1} \mid \hat{\mathbf{o}}_{t+1}, \mathbf{m}), \tag{3a}$$

$$\hat{\mathbf{o}}_{t+1} = \ \kappa(\mathbf{o}_{t-\tau:t}), \ \mathbf{o}_{t-\tau:t} = \ \psi(\mathbf{y}^R_{t-\tau:t}), \tag{3b}$$

$$\mathbf{m} = g(\mathbf{y}^R_{t-\tau:t}), \tag{3c}$$

$$\mathbf{y}^R_{t-\tau:t} = \Lambda(\mathbf{d}_{t-\tau:t}). \tag{3d}$$

In this framework, (3a) is the VAE predictor module from which we sample future maps $\mathbf{o}_{t+1}$. (3b) represent the dynamic object module, where $\kappa(\cdot)$ processes the time series data for

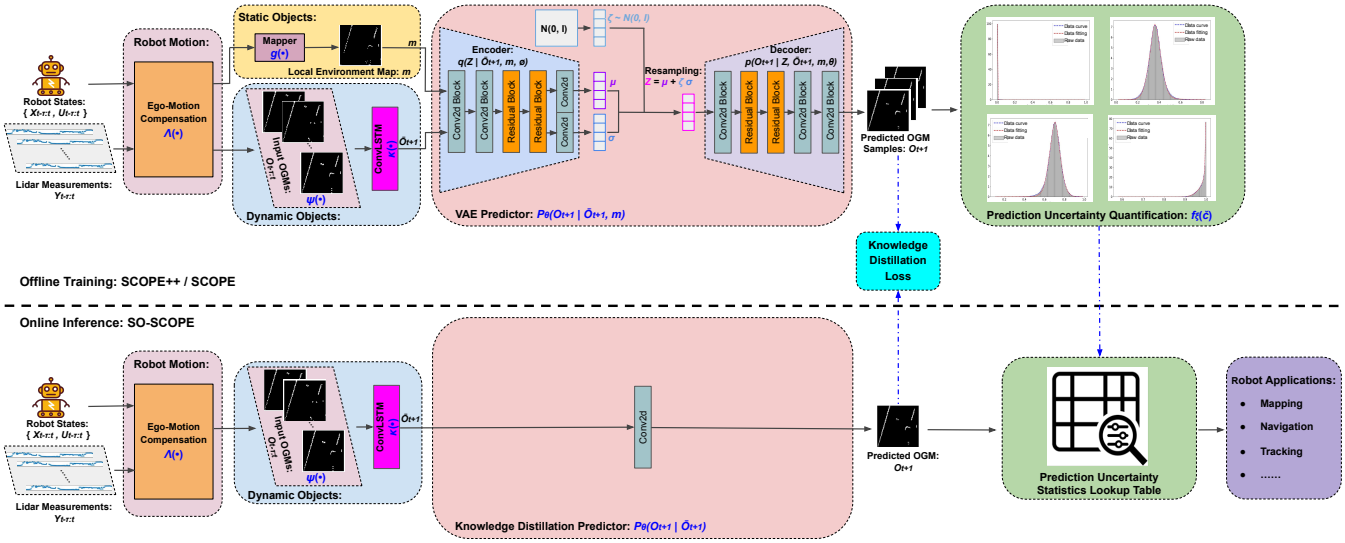


Fig. 2. System architectures of the SCOPE++ predictor, SCOPE predictor, and its software-accelerated SO-SCOPE predictor (note that SCOPE omits the Static Objects block compared to SCOPE++). The basic process of the SCOPE++ predictor is: 1) based on a history of robot states, the robot transfers the lidar measurement history to the predicted coordinate frame of the robot to compensate for the ego-motion, 2) these compensated lidar measurements are used to generate a local environment map to account for static objects, and a set of OGMs to account for dynamic objects, and 3) the local map of static objects and the predicted OGM of dynamic objects are fed into an variational autoencoder to predict the future OGM. To accelerate the SCOPE++ predictor, we first follow the SCOPE network architecture and replace the VAE network with a single convolutional layer, then use knowledge distillation technology to train the SO-SCOPE network to obtain the prediction information, and finally, we model and quantify the prediction uncertainty of the SCOPE++ to obtain uncertainty statistics and use them to generate uncertainty estimates of SO-SCOPE.

dynamic objects, $\psi(\cdot)$ is the conversion function like (2b), and R denotes the local coordinate frame of the robot at predicted time $t+n$. (3c) is the static objects module, where $g(\cdot)$ is the occupancy grid mapping function for static objects. Finally, (3d) is the robot motion module, where $\Lambda(\cdot)$ is the transformation function to compensate for robot motion.

Note that (3) only predicts future states at the next time step $t+1$ and is used for training. To predict a multi-step future state at time $t+n$, we can easily utilize the autoregressive mechanism and feed the next-step prediction back $\mathbf{o}_{t+1}$ to our SCOPE/SCOPE++ network (3a) for $n-1$ time steps to predict the future states at time step $t+n$. Note that the prediction horizon n could theoretically be any time step.

1) Robot Motion Compensation $\Lambda(\cdot)$: To account for robot motion, we propose a simple and effective ego-motion compensation mechanism. We first predict the future pose of the robot at the end of our prediction horizon, time $t+n$. To do this, we use a constant velocity motion model, as it is the most widely used motion model for tracking [62] and often outperforms more state-of-the-art methods in general settings [63] when used on a relatively short period (*i.e.,* less than 1 s). Note that other more appropriate robot motion models tailored to specific robot models can be used to provide better ego-motion compensation. This results in a predicted pose for the robot $\mathbf{x}_{t+n}$, and we use this pose as the origin of our coordinate frame R (where the x-axis is forward and the y-axis is left, as is standard in mobile robotics [64]). We then transform all data into this frame, resulting in poses $\mathbf{x}^R_{t-\tau:t}$ and lidar scans $\mathbf{y}^R_{t-\tau:t}$. Using this data, we can generate the sequence of OGMS $\mathbf{o}^R_{t-\tau:t}$ using the conversion function $\psi(\cdot)$ by setting each lidar scan "hit" to 1 and all other cells to 0 (for more details, see [4]). The benefit of ego-motion compensation is that we can treat these lidar measurements from a moving lidar sensor as observations in common world-fixed frame R, which significantly reduces the difficulty of OGM predictions and improves accuracy. Note this only requires the robot to have accurate odometry over the history window τ (typically on the order of 1 s), but not accurate global localization.

2) Dynamic Object Prediction $\kappa(\cdot)$: Tracking and predicting dynamic objects such as pedestrians is the hardest part of environmental prediction in complex dynamic scenes. It requires some techniques to process a set of time series data to capture the motion information. While the traditional particle-based methods [16] require explicitly tracking objects and treat each grid cell as an independent state, recent learning-based methods [8], [17]–[20], [22]–[24], [27] prefer to use RNNs to directly process the observed time series OGMs. Based on these trends, we choose the most popular ConvLSTM unit to process the spatiotemporal OGM sequences $\mathbf{o}_{t-\tau:t}$. However, while other works [7], [18] explicitly decouple the dynamic and static/unknown objects and use different networks to process them separately, we argue that the motion of dynamic objects is related to their surroundings, and that explicit disentangling may lose some useful contextual information. For example, pedestrians walking through a narrow corridor are less likely to collide with or pass through surrounding walls. To exploit the useful contextual information between dynamic objects and their surrounding, we directly feed the observed OGMs $\mathbf{o}^R_{t-\tau:t}$ into a ConvLSTM unit $\kappa(\cdot)$ and implicitly predict the future state $\hat{\mathbf{o}}_{t+1}$ of dynamic (and static) objects.[2]

[2]Note that since the VAE is designed to provide uncertainty estimates of future states (while the static object module is designed to provide static information), our supervised learning mechanism will implicitly regularize the output of the ConvLSTM unit to contain the future states of dynamic objects.

TABLE I
RUNNING PERFORMANCE PERCENTAGE OF SCOPE++ PREDICTOR ON THE EMBEDDED DEVICE

Metrics	Ego-Motion Compensation Module	Static Object Module	Dynamic Object Module	VAE Module
Runtime	9.04%	**53.41%**	20.93%	16.62%
Memory Usage	0.32%	1.28%	48.38%	**50.02%**

3) Static Object Segmentation $g(\cdot)$: While predicting dynamic objects plays a key role in environmental prediction, paying extra attention to static objects is also important to improve prediction accuracy. The main reason is that the area occupied by static objects is much larger than that of dynamic objects, as shown in Fig. 2, where static objects such as walls are much larger than dynamic objects such as pedestrians, which are sparse and scattered point clusters. Another reason is that static objects maintain their shape and position over time, contributing to the scene geometry and giving a global view of the surroundings. To account for static objects, we utilize a local static environment map $\mathbf{m}$ as a prediction for future static objects, which is a key contribution of our work. We generate this local environment map $\mathbf{m}$ using a GPU-accelerated implementation[3] of the standard inverse sensor model [65] for $g(\cdot)$ that parallelizes independent cell state update operations by taking advantage of the additive update rule (for more details, see [4]). This Bayesian approach generates a robust estimate of the local map, where dynamic objects are treated as noise data and removed over time.

4) VAE Predictor: The dynamic and static prediction functions from (3b) and (3c) yield one estimate of the future environment. However, it is unlikely that this exact map will be correct. To account for this, we wish to generate a range of possible future environment states that capture the uncertainty of the environment. We do this using a variational autoencoder (VAE), a generative machine learning technique that creates samples that are "similar" to the predicted OGM $\hat{\mathbf{o}}_{t+1}$.

To represent the stochasticity of environment states, we assume that environment states $\mathbf{o}_{t-\tau:t+n}$ are generated by some unobserved, random, latent variable $\mathbf{z}$ that follow a prior distribution $p_\theta(\mathbf{z})$. Our VAE model (3a) can be rewritten as

$$p_\theta(\mathbf{o}_{t+1} \mid \hat{\mathbf{o}}_{t+1}, \mathbf{m}) = \int p_\theta(\mathbf{z}) p_\theta(\mathbf{o}_{t+1} \mid \mathbf{z}, \hat{\mathbf{o}}_{t+1}, \mathbf{m}) dz. \quad (4)$$

Since we are unable to directly optimize this marginal likelihood and obtain optimal parameters θ, we use a VAE network to parameterize our prediction model $p_\theta(\mathbf{o}_{t+1} \mid \hat{\mathbf{o}}_{t+1}, \mathbf{m})$, outlined in Fig. 2, where the inference network (encoder) parameterized by ϕ refers to the variational approximation $q_\phi(\mathbf{z} \mid \hat{\mathbf{o}}_{t+1}, \mathbf{m})$, the generative network (decoder) parameterized by θ refers to the likelihood $p_\theta(\mathbf{o}_{t+1} \mid \mathbf{z}, \hat{\mathbf{o}}_{t+1}, \mathbf{m})$, and the standard Gaussian distribution $\mathcal{N}(0, 1)$ refers to the prior $p_\theta(\mathbf{z})$. Then, from work [66], we can simply maximize the evidence lower bound (ELBO) loss $\mathcal{L}(\theta, \phi; \mathbf{o}_{t+1})$ to optimize this marginal likelihood and get the optimal θ:

$$\begin{aligned} \mathcal{L}(\theta, \phi; \mathbf{o}_{t+1}) =& \mathbb{E}_{q_\phi(\mathbf{z}|\hat{\mathbf{o}}_{t+1}, \mathbf{m})} \left[\log p_\theta(\mathbf{o}_{t+1} \mid \mathbf{z}, \hat{\mathbf{o}}_{t+1}, \mathbf{m}) \right] \\ & - KL\big(q_\phi(\mathbf{z} \mid \hat{\mathbf{o}}_{t+1}, \mathbf{m}) \,\|\, p_\theta(\mathbf{z})\big). \end{aligned} \quad (5)$$

[3]https://github.com/TempleRAIL/occupancy_grid_mapping_torch

The first term on the right-hand side (RHS) is the expected generative error, describing how well the future environment states can be generated from the latent variable $\mathbf{z}$. The second RHS term is the Kullback–Leibler (KL) divergence, describing how close the variational approximation is to the prior.

We use mini-batching, Monte-Carlo estimation, and reparameterization tricks to calculate the gradients of the ELBO (5) [66], and obtain the optimized model parameters ϕ and θ. Finally, our VAE can integrate the predicted features $\{\hat{\mathbf{o}}_{t+1}, \mathbf{m}\}$ of dynamic and static objects, and output a probabilistic estimate of future OGM states with uncertainty awareness.

IV. SOFTWARE OPTIMIZATION

We demonstrated in our previous work [4] (results also in Section V) that SCOPE and SCOPE++ can accurately predict OGMs. However, the use of multiple neural networks is resource-intensive. For example, the VAE used to generate samples to provide uncertainty estimates is so memory-intensive that we can only generate 8 samples using an NVIDIA Jetson AVG Xavier embedded device [4].[4] To make our proposed SCOPE/SCOPE++ work efficiently in resource-constrained robots, we decide to optimize them on the software side.

A. Resource Utilization

Table I details the running performance percentage of each module in the SCOPE++ predictor (generating 8 samples) in terms of runtime and memory usage on our embedded computer. We can easily see that the runtime is mainly limited by the static and dynamic object modules (with the ConvLSTM unit), while the memory usage is mainly limited by the dynamic object (with ConvLSTM unit) and VAE modules. This analysis explains why our proposed SCOPE predictor without the static object module is much faster than our SCOPE++ predictor.

Since the key ConvLSTM unit in the dynamic object module is already an accelerated RNN unit (compared to the Vanilla RNN unit), it is difficult to further optimize this module. Therefore, we choose to perform software optimization on the static object module and VAE module to increase running speed and reduce memory consumption while maintaining the accurate prediction and stochasticity of our SCOPE/SCOPE++.

According to Table I and our network architecture design, a straightforward method is to remove the static object module (*i.e.,* use SCOPE instead of SCOPE++), especially since the two variants achieve comparable prediction accuracy. Next, we will look at replacing the VAE network by utilizing knowledge distillation techniques to compress our SCOPE network to

[4]Generating 8 samples is a trade-off in running speed and memory consumption when running with other navigation-related packages such as `amcl`, `move_base`, and other sensor driver ROS packages.

maintain its prediction performance, and then quantify the prediction uncertainty of the SCOPE to provide uncertainty estimates to maintain its stochastic nature.

B. Knowledge Distillation

Compared with other deep neural network compression techniques (*e.g.,* pruning, quantization), knowledge distillation techniques can provide better generalization capabilities and allow the selection of different network architectures [67]. To design a simple and efficient student network for knowledge distillation, we remove the static object module (following our fast SCOPE network architecture) and replace the complete VAE network with a single convolutional layer, which can be seen from the SO-SCOPE part of Fig. 2. Therefore, our SO-SCOPE network is very simple and efficient, consisting of only one ConvLSTM layer and one convolutional layer.

We use the SCOPE instead of the SCOPE++ as our teacher network for knowledge distillation, shown in Fig. 2. There are two reasons why we choose the faster and less accurate SCOPE network rather than the complex and more accurate SCOPE++ network as the teacher network. First, as Cho et al. [46] suggested, larger or more accurate models generally do not make better teachers in the knowledge distillation, especially when the student's ability is too low to successfully imitate the teacher. Second, Beyer et al. [68] suggested that keeping "consistent" teaching can improve knowledge distillation. Our SO-SCOPE student network follows the SCOPE network design, which is almost the same as the SCOPE network except that a single convolutional layer is used instead of the VAE network (*i.e.,* the input view of the convolutional layer is consistent with the VAE network). Therefore, this structural similarity and consistency improves knowledge transfer from the teacher network to the student network. This will accelerate the running speed and also reduce the memory consumption.

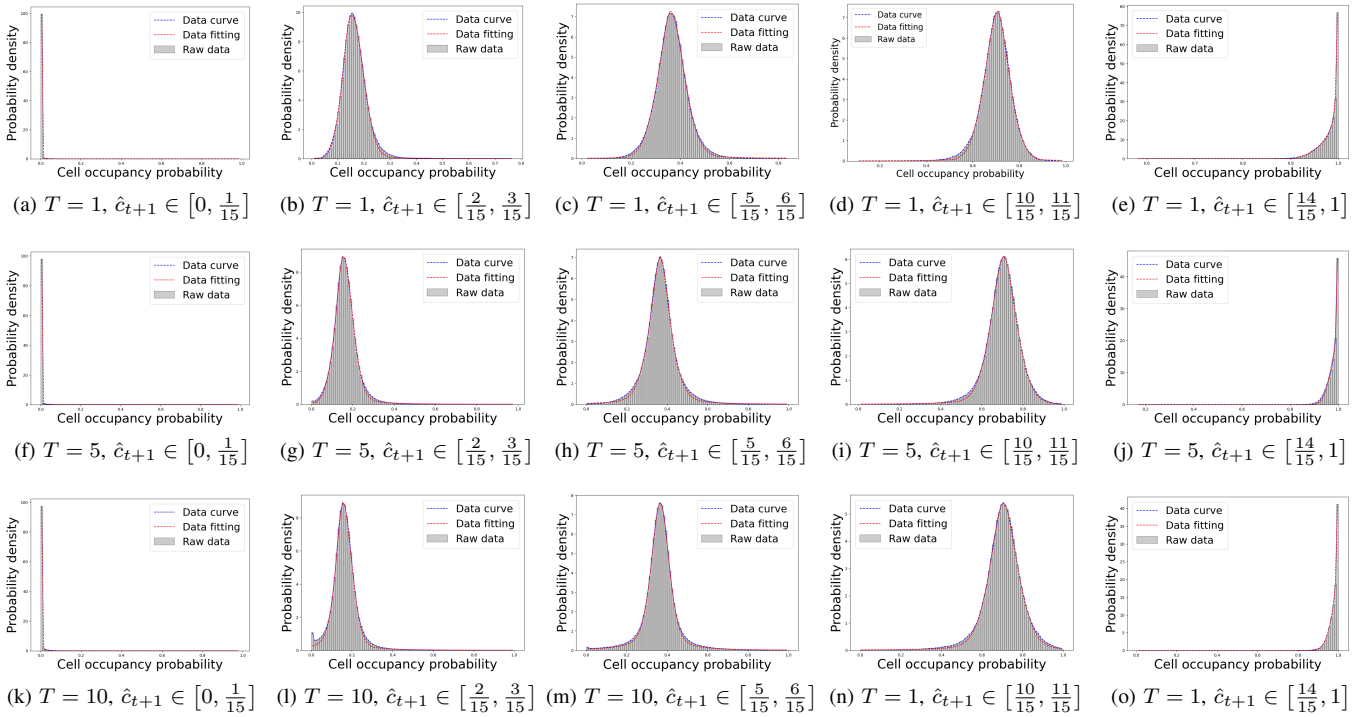
To train our SO-SCOPE student network, we directly use the output of our trained SCOPE teacher model in the OGM-Turtlebot2 training dataset (see Section V-A) as "soft" labels instead of using ground truth labels, which follow the key idea of knowledge distillation. To speed up training, we use 1 random sample output from SCOPE as a "soft" label in each training iteration. Note that since our SCOPE is stochastic and generates different samples even with the same input, this stochastic sample output expands the original training dataset (*i.e.,* data augmentation) and further improves the generalization ability of the SO-SCOPE. We use the binary cross entropy (BCE) loss function to train our SO-SCOPE student network and get our final SO-SCOPE predictor to provide deterministic OGM predictions. After completing the knowledge distillation training, we directly use the output of the SO-SCOPE predictor to provide OGM prediction information. However, this distilled network is now deterministic, *i.e.,* each input will generate a single output. Our next step will address this limitation.

C. Uncertainty Quantification

To modify SO-SCOPE, we will characterize the statistics of the VAE from our trained SCOPE++ predictor and then combine this data with the output of the distillation network (which should generate an OGM that is similar to an output of the VAE) to draw samples with identical statistics to our original network but in a much more memory- and time-efficient manner. Note that using SCOPE++ here is a reasonable choice because our SCOPE++ predictor provides a more accurate and comprehensive uncertainty estimate than SCOPE. To characterize the VAE, we collected $8,000$ different input sequences to the ConvLSTM from the OGM-Turtlebot2 testing dataset (see Section V-A) and generated 32 VAE samples for each input, giving us a total of $256,000$ output samples for each prediction horizon $T = 1, \ldots, \tau$. Although using the OGM-Turtlebot2 testing dataset may raise the issue of data leakage, we used the testing dataset to characterize the VAE because: 1) SCOPE++ is well trained on the training dataset and, as a result, the uncertainty of SCOPE++'s output in the training dataset is very small and the uncertainty statistics in the training dataset do not reflect the true uncertainty statistics of the SCOPE++; in contrast, the uncertainty statistics in the testing dataset reflect the true uncertainty statistics of the SCOPE++ very well; and 2) the key aim of proposing our software-optimized SO-SCOPE predictor is to optimize and accelerate the trained resource-intensive SCOPE/SCOPE++ predictors to allow them to run in real time on resource-constrained robots. It is reasonable to choose a testing dataset to characterize uncertainty statistics because we already have the predictors that need to be accelerated and we can easily use them to collect real data statistics from the application environment.

Our analysis relies on two factors. First, in an OGM, the probability of occupancy for each cell is independent of all other cells [65]. In other words, for two cells c and c', then $p(c \text{ is occupied and } c' \text{ is occupied}) = p(c \text{ is occupied})p(c' \text{ is occupied})$. Second, based on our use of a VAE, we make the assumption that the distribution of occupancy probabilities in the VAE output depends on the occupancy probability of the input. This is reasonable since a VAE is trained to have similar outputs near to one another in the learned latent space. Let $\hat{c}_{t+T} = p(c \text{ is occupied at time } t+T)$ denote the probability of occupancy of cell c predicted by the ConvLSTM layer T steps into the future (*i.e.,* we use the average of the occupancy probabilities predicted by the VAE for the same cell as its estimate) and $\tilde{c}_{t+T}$ be the probability of occupancy predicted for that same cell by the VAE. Then we assume $p(\tilde{c}_{t+T} \mid \hat{c}_{t+T}) \neq p(\tilde{c}_{t+T})$.

Based on these factors, we will examine trends in the VAE probabilities $\tilde{c}_{t+T}$ (for $T = 1, \ldots, \tau$) conditioned on the value of $\hat{c}_{t+T}$. We use 15 bins for $\hat{c}_{t+T}$, evenly spaced from 0 to 1 (since the probability range of OGM cell is in $[0, 1]$). With these binned OGM cell samples, we visualize the distribution of $\tilde{c}_{t+T}$ for each $\hat{c}_{t+T}$ bin, as shown in Fig. 3. We can see a few useful trends in the data. First, cells that are predicted to be empty by the ConvLSTM (*i.e.,* in bin 0) remain empty for nearly all VAE samples. This trend holds during the full prediction window, with the occupancy probability increasing only slightly as T becomes larger. Second, the peak of the distribution for $\tilde{c}_{t+T}$ remains very close to the value of $\hat{c}_{t+1}$. This implies that the VAE generates a consistent distribution around the ConvLSTM output. Third, heavier tails in the distributions as T increases. This makes sense as a longer horizon into the future leads



(a) $T = 1$, $\hat{c}_{t+1} \in \left[0, \frac{1}{15}\right]$ (b) $T = 1$, $\hat{c}_{t+1} \in \left[\frac{2}{15}, \frac{3}{15}\right]$ (c) $T = 1$, $\hat{c}_{t+1} \in \left[\frac{5}{15}, \frac{6}{15}\right]$ (d) $T = 1$, $\hat{c}_{t+1} \in \left[\frac{10}{15}, \frac{11}{15}\right]$ (e) $T = 1$, $\hat{c}_{t+1} \in \left[\frac{14}{15}, 1\right]$

(f) $T = 5$, $\hat{c}_{t+1} \in \left[0, \frac{1}{15}\right]$ (g) $T = 5$, $\hat{c}_{t+1} \in \left[\frac{2}{15}, \frac{3}{15}\right]$ (h) $T = 5$, $\hat{c}_{t+1} \in \left[\frac{5}{15}, \frac{6}{15}\right]$ (i) $T = 5$, $\hat{c}_{t+1} \in \left[\frac{10}{15}, \frac{11}{15}\right]$ (j) $T = 5$, $\hat{c}_{t+1} \in \left[\frac{14}{15}, 1\right]$

(k) $T = 10$, $\hat{c}_{t+1} \in \left[0, \frac{1}{15}\right]$ (l) $T = 10$, $\hat{c}_{t+1} \in \left[\frac{2}{15}, \frac{3}{15}\right]$ (m) $T = 10$, $\hat{c}_{t+1} \in \left[\frac{5}{15}, \frac{6}{15}\right]$ (n) $T = 1$, $\hat{c}_{t+1} \in \left[\frac{10}{15}, \frac{11}{15}\right]$ (o) $T = 1$, $\hat{c}_{t+1} \in \left[\frac{14}{15}, 1\right]$

Fig. 3. Uncertainty quantification of the predicted OGM cell bins (each column is one occupancy bin $\tilde{c}_{t+1}$) at different prediction time steps (each row is one timestep T). Grey histograms are the raw predicted OGM cell data of our SCOPE++ predictor. Blue curves are the connecting curves of these raw predicted OGM cell data. Red curves are the fitting curves from our proposed mixture model of truncated normal and skew-Cauchy distributions in (6).

to further uncertainty and further deviation from the current time. Fourth, the distributions skew towards 0 as the prediction horizon T increases. This makes sense as cells are more likely to become empty as people move about in the scene. Fifth, for situations where $\hat{c}_{t+1}$ is low, we see a peak in the distribution of $\tilde{c}_{t+T}$ at 0. This is caused by uncertain objects moving out of a cell, leaving it empty.

Our goal is to fit a model to the resulting histograms. Our model must be able to capture the heavy tails, skew, and the peak near $\tilde{c} = 0$. Based on this, we use a mixture model of a truncated normal and a skew-Cauchy distribution to describe the distribution of $\tilde{c}_{t+T}$ values, which can be expressed as:

$$f_\xi(\tilde{c}) = w \frac{\sqrt{\frac{2}{\pi\sigma_{\text{tn}}^2}} \exp\left(-\frac{1}{2}\left(\frac{\tilde{c}-\mu_{\text{tn}}}{\sigma_{\text{tn}}}\right)^2\right)}{\text{erf}\left(\frac{b-\mu_{\text{tn}}}{\sqrt{2}\sigma_{\text{tn}}}\right) - \text{erf}\left(\frac{a-\mu_{\text{tn}}}{\sqrt{2}\sigma_{\text{tn}}}\right)} + (1-w)\frac{1}{\sigma_{sc}\pi\left[1+\left(\frac{|\tilde{c}-\mu_{sc}|}{\sigma_{sc}(1+\lambda\,\text{sign}(\tilde{c}-\mu_{sc}))}\right)^2\right]}, \tag{6}$$

where the first term on RHS is the truncated normal distribution (with mean μ_{tn}, standard deviation σ_{tn}, lower bound a, and upper bound b) and the second RHS term is the skew-Cauchy distribution (with location parameter μ_{sc}, scale parameter σ_{sc}, and skewness λ), and $w \in [0, 1]$ is the mixture weight.

Let $\xi = [w, a, b, \mu_{\text{tn}}, \sigma_{\text{tn}}, \lambda, \mu_{\text{sc}}, \sigma_{\text{sc}}]$ be the vector of mixture model parameters. We use a least square optimization algorithm [69] to fit ξ to our predicted OGM cell bin data. Figure 3 shows the results of this fit. We see that the model achieves strong performance across a range of bins and timesteps, indicating that our methods work well.

The end result of our analysis above is a set of 150 ξ vectors, one for each of the 15 $\hat{c}$ bins and each of the 10 time steps T. For each set of mixture model parameters, we can generate different types of distribution statistics (*e.g.,* mean, median, mode, entropy, and random variate) and store these values in a lookup table. We call this lookup table the prediction uncertainty statistics lookup table.

Now that we have accurate models for the statistics of how each cell in the map will change over time, we can integrate that information into the SO-SCOPE model. To do this, we use the output of the distillation layer to get $\hat{c}_{t+T}$ for each cell in the OGM $\mathbf{o}_{t+T}$, as shown in the SO-SCOPE part of Fig. 2. We can then use the uncertainty statistics lookup table to get statistics of that cell, or we can use the cumulative distribution function of (6) to draw samples $\tilde{c}_{t+T}$. This allows our SO-SCOPE model to account for uncertainty in the predicted future using knowledge distilled from the SCOPE and SCOPE++ models to achieve comparable accuracy with significantly smaller computational effort and memory consumption.

V. OGM Prediction Experiments and Results

To demonstrate the prediction performance of our proposed approaches, we first test our algorithms on a simulated dataset and two public real-world sub-datasets from the socially compliant navigation dataset (SCAND) [70]. We comprehensively evaluate the inference speed and memory usage of our proposed predictors on a resource-constrained embedded computing device to show that our software-optimized predictor

can achieve surprising performance improvements. We also characterize the uncertainty of our proposed predictors across different sample sizes and numbers of objects to demonstrate the diversity and consistency of their uncertainty estimates.

A. Datasets

We used three publicly available OGM datasets to evaluate our proposed prediction algorithms and baselines, one self-collected from the Gazebo simulator (*i.e.,* OGM-Turtlebot2) [14] and two extracted from the real-world dataset SCAND [70] (*i.e.,* OGM-Jackal and OGM-Spot).

1) OGM-Turtlebot2 Dataset: To train our predictors and baselines, we collected the OGM-Turtlebot2 dataset using the 3D robot-pedestrian interaction Gazebo simulator from our previous work [11], [13]. We collected the robot states $\{\mathbf{x}, \mathbf{u}\}$ and raw lidar measurements $\mathbf{y}$ at a sampling rate of 10 Hz. We collected a total of 94,891 $\mathbf{d} = \{\mathbf{x}, \mathbf{u}, \mathbf{y}\}$ data tuples, dividing this into three separate subsets for training (67,000 tuples), validation during training (10,891 tuples), and final testing (17,000 tuples). More dataset details can be found in [4].

2) OGM-Jackal and OGM-Spot Datasets: The OGM-Jackal and OGM-Spot datasets were processed and created from the original SCAND dataset [70], which was collected by humans manually controlling the Jackal robot and the Spot robot around the indoor/outdoor flat environments at UT Austin.[5] More dataset processing details can be found in [4].

B. OGM Prediction

We compare our proposed SCOPE, SCOPE++, and SO-SCOPE algorithms with six deep learning-based baselines: ConvLSTM [5], DeepTracking [6], PhyDNet [7], SAAConvLSTM [8], TAAConvLSTM [8], and LOPR [9], and six ablation baselines: SCOPE-NEMC (SCOPE with No Ego-Motion Compensation module), SO-SCOPE-GTEMC (SO-SCOPE with Ground Truth of Ego-Motion Compensation module), SO-SCOPE-MEAN (use the mean value from the prediction uncertainty statistics lookup table as the OGM prediction), SO-SCOPE-MEDIAN (use the median from the lookup table), SO-SCOPE-MODE (use the mode from the lookup table), and SO-SCOPE-SAMPLE (draw a random sample from distribution) [6]. Note that SCOPE-NEMC is used to demonstrate the advantages of our proposed ego-motion compensation module and the network structure compared to other image-based baselines without ego-motion compensation. SO-SCOPE-GTEMC is used to demonstrate the upper-performance limit of the SO-SCOPE predictor with accurate ego-motion compensation. All these networks were implemented using PyTorch framework [71] and trained using *only* our self-collected OGM-Turtlebot2 dataset.

[5]Force control for complex dynamic motions like the Spot robot is handled by the low-level controller, while the human motion control commands collected in the dataset are high-level 2D velocity control commands.

[6]Note that we ignore two ablation baselines of training the same SO-SCOPE network using dataset labels and using the SCOPE++ model as the "teacher" network, as they are both used to support the knowledge distillation theory and validated by previous researchers [46], [67], [68].

1) Evaluation Metrics: We use three metrics: 1) weighted mean square error (WMSE) [72] to evaluate absolute error between the ground truth and predicted maps, 2) structural similarity index measure (SSIM) [73] to measure the similarity between the two maps, and 3) optimal subpattern assignment metric (OSPA) [74] to measure the difference in the number and locations of distinct objects within the maps. Note that OGM prediction works [8], [9], [17]–[20], [22]–[24], [27] only use the computer vision metrics (*e.g.,* MSE, F1 Score, and SSIM) to evaluate the quality of predicted OGMs. Since a blurred prediction with uncertainty may perform better in some cases, we are the first to evaluate the predicted OGMs from the perspective of multi-target tracking (*i.e.,* OSPA distance), which we believe gives a more physically meaningful evaluation than image quality as it accounts for the number of and locations of objects in the scene. More details can be found in [4].

2) Qualitative Results: Figure 4 and the accompanying multimedia illustrate the future OGM predictions generated by our proposed predictors and the baselines. We observe three interesting phenomena. First, the image-based baselines, especially the PhyDNet, generate blurry future predictions after 5-time steps, with only blurred shapes of static objects (*i.e.,* walls) and missing dynamic objects (*i.e.,* pedestrians). We believe this is because these six baselines are deterministic models that use less expressive network architectures, only treat time series OGMs as images/video, and cannot capture and utilize the kinematics and dynamics of the robot itself, dynamic objects, and static objects. Second, the SCOPE++ with a local environment map has a sharper and more accurate surrounding scene geometry (*i.e.,* right walls) than the SCOPE without it. This difference indicates that the local environment map for static objects is beneficial and plays a key role in predicting surrounding scene geometry. Third, our proposed software-optimized SO-SCOPE can achieve clear and sharp OGM predictions similar to its "teacher" SCOPE, which demonstrates the effectiveness of applying knowledge distillation techniques to optimize our SCOPE/SCOPE++.

3) Quantitative Results: While achieving sharper predictions is encouraging, we also need quantitative measures. We test 14 OGM predictors on the OGM-Turtlebot2 test dataset, OGM-Jackal dataset, and OGM-Spot dataset and provide a comprehensive benchmark by evaluating absolute error, structure similarity, and tracking accuracy. Recall that we only use the OGM-Turtlebot2 during training, so the other two datasets are entirely new and were generated by robots with different speeds and kinematics (*i.e.,* Spot ambulates).

a) Absolute Error: To evaluate the absolute error of predicted OGMs, we calculate the average WMSE of predicted OGMs for the next 10 prediction time steps, outlined in Figs. 5a to 5c. It can be seen that our proposed SCOPE family of methods with ego-motion compensation achieves significantly better average WMSE than the SCOPE-NEMC ablation baseline without ego-motion compensation in all test datasets, which illustrates the effectiveness of ego-motion compensation. The average WMSEs of our proposed SCOPE family of methods at different prediction time steps are not significantly different from one another, but are all lower than the six image-based baselines (*i.e.,* ConvLSTM [5], DeepTracking [6], PhyDNet [7],

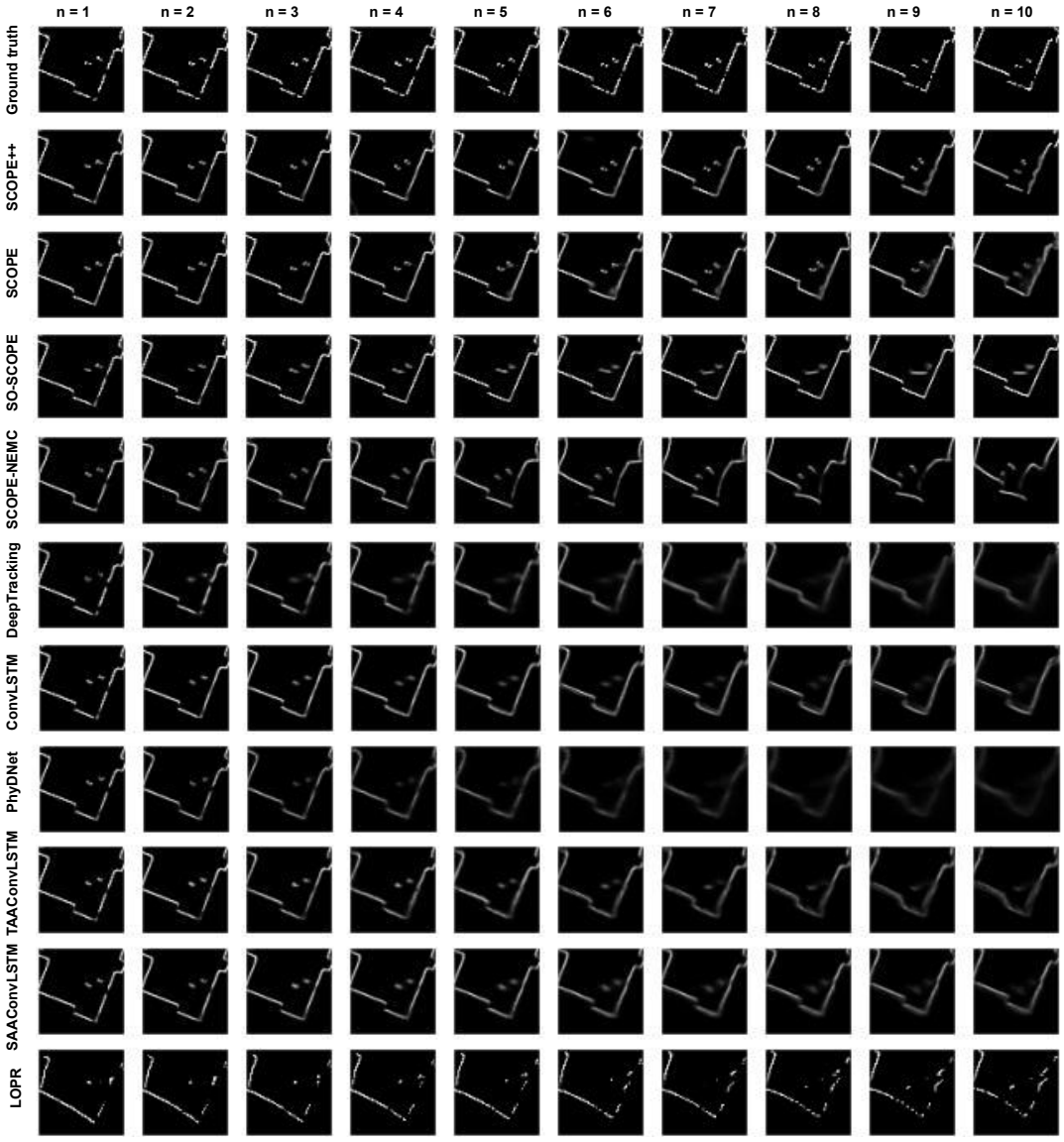
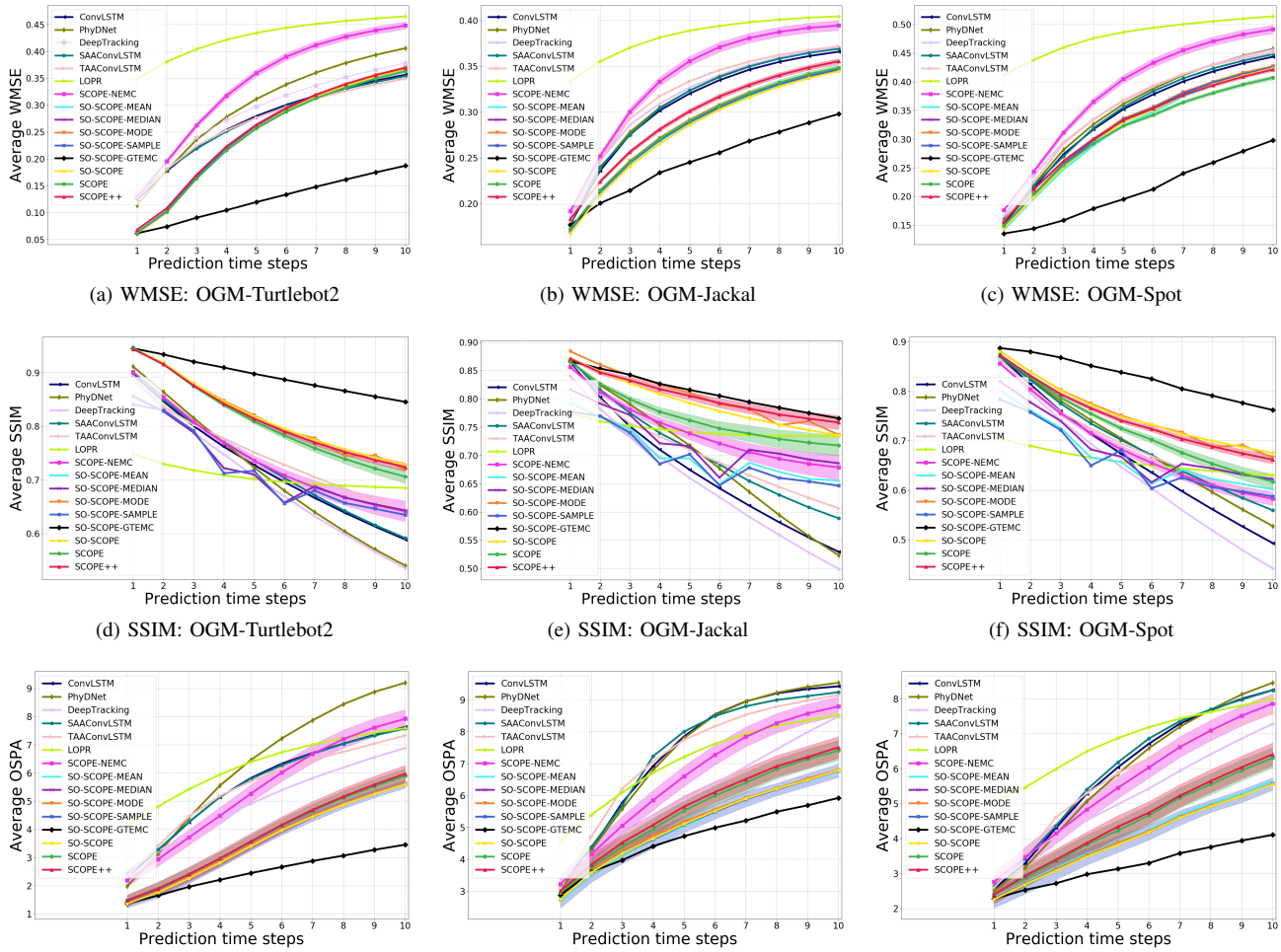


Fig. 4. Prediction showcase of ten OGM predictors tested on the OGM-Turtlebot2 dataset. The black and white areas are free and occupied space respectively.

SAAConvLSTM [8], TAAConvLSTM [8], and LOPR [9]). This trend holds across all three test datasets collected by different robots. This shows that the proposed SCOPE family of methods utilizing kinematic and dynamic information can predict more accurate OGMs than image-based approaches at different prediction time steps. Furthermore, the average WMSEs of the SO-SCOPE-GTEMC are much higher than that of the SO-SCOPE, which indicates that the best prediction performance can be achieved if our SO-SCOPE is equipped with an accurate customized ego-motion compensation algorithm for the robot instead of using only a general constant velocity motion model.

b) Structure Similarity: To evaluate the structure similarity of predicted OGMs, we calculate the average SSIM of predicted OGMs for the next 10 prediction time steps, as shown in Figs. 5d to 5f. Unlike general WSME, which cares about the absolute error of a single OGM cell, SSIM cares about the entire OGM structure and its scene geometry. First, we can see that the average SSIMs of our proposed SCOPE, SCOPE++, and SO-SCOPE predictors at different prediction time steps are significantly higher than that of SCOPE-NEMC, and the average SSIMs of SCOPE-NEMC are almost higher than the other six image-based baselines, which further illustrates the effectiveness of ego-motion compensation and also the advantage of our network design. Second, the average SSIMs of our software-optimized SO-SCOPE predictor and its statistical ablation baseline SO-SCOPE-MODE (*i.e.,* using peak values as cell prediction values) are not significantly different (similar to that of the SCOPE++ predictor), but much higher than the other three statistical ablation baselines (*i.e.,* SO-SCOPE-MEAN, SO-SCOPE-MEDIAN, SO-SCOPE-SAMPLE). It shows that the direct output of SO-SCOPE "student" network can represent the prediction information and the mode from our prediction uncertainty statistics lookup table is also a good statistic to represent the prediction information (compared to other statistics like mean, median, and random variate), which further supports the choice of using knowledge distillation techniques



(a) WMSE: OGM-Turtlebot2 (b) WMSE: OGM-Jackal (c) WMSE: OGM-Spot

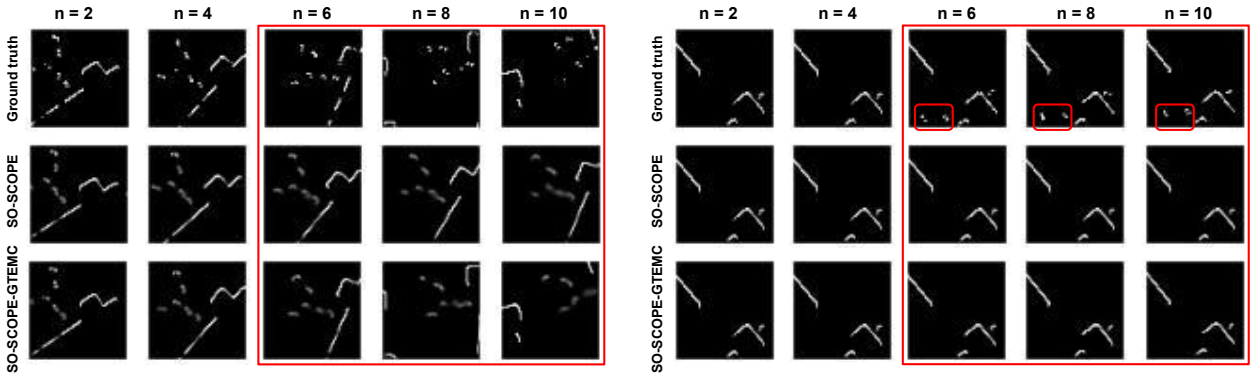
(d) SSIM: OGM-Turtlebot2 (e) SSIM: OGM-Jackal (f) SSIM: OGM-Spot

(g) OSPA: OGM-Turtlebot2 (h) OSPA: OGM-Jackal (i) OSPA: OGM-Spot

Fig. 5. WMSE (row 1), SSIM (row 2), and OSPA (row 3) for all tested methods on our 3 different datasets (columns). Each figure shows how the average value (over samples in the test datasets) of a given metric changes as a function of the prediction horizon, where lower is better for WMSE and OSPA and higher is better for SSIM. Lines for SCOPE-NEMC, SCOPE, and SCOPE++ include a 95% confidence interval over 32 samples drawn from the VAE module.

and uncertainty quantification to software optimize our original SCOPE/SCOPE++ predictors. Third, it is interesting that the average SSIM of our SCOPE++ with a local environment map is significantly higher than that of SCOPE without a local environment map in long-term predictions over multiple time steps. This indicates that the local environment map that takes into account static objects helps to predict the long-term future of the environment. This is reasonable because the local environment map describes the basic scene geometry and shape. Finally, the highest average SSIMs of SO-SCOPE-GTEMC also show the upper limit of the performance of our SO-SCOPE.

c) Tracking Accuracy: To further evaluate the tracking accuracy of predicted OGMs, we calculate the average OSPA of predicted OGMs for the next 10 prediction time steps, as shown in Figs. 5g to 5i. While the average WMSE and SSIM are the evaluation metrics from computer vision, the average OSPA is from the multi-target tracking and is more suitable for evaluating the physical quality of OGMs. Figures 5g to 5i reveals that the average OSPA errors of our proposed SCOPE family of predictors are significantly lower than that of SCOPE-NEMC, and the average OSPA errors of SCOPE-NEMC are almost lower than that of other image-based baselines. This exciting result further demonstrates the preferential performance of our proposed motion-based methods (*i.e.,* SCOPE, SCOPE++, and SO-SCOPE) in tracking or predicting environmental states (*i.e.,* localization and cardinality) outperform SCOPE-NEMC, and while SCOPE-NEMC almost outperforms other image-based baselines (similar to the SSIM metric). Furthermore, the average OSPA errors of our software-optimized SO-SCOPE predictor and its statistical ablation baselines are not significantly different from one another (similar to the WMSE metric), but are lower than the original SCOPE and SCOPE++ predictors (especially in the real-world OGM-Jackal and OGM-Spot datasets), which is attributed to the use of knowledge distillation techniques to distill stochastic neural network models (*i.e.,* data augmentation). It is reasonable and consistent with the argument in many knowledge distillation works [43], [67] that the "student" network can generalize better than the "teacher" network. However, the average OSPA errors of our proposed SCOPE and SCOPE++ predictors are almost the same, where the local environment map for static objects does not help SCOPE++ to reduce the OSPA error. We believe



(a) Inaccurate robot motion compensation

(b) Occlusion or sudden appearance

Fig. 6. Two typical prediction failure cases of the SCOPE series predictors (SO-SCOPE as the example). The red bounding boxes highlight prediction errors.

TABLE II
INFERENCE SPEED, MODEL SIZE, AND MEMORY USAGE OF DIFFERENT OGM PREDICTORS

Models	ConvLSTM [5]	PhyDNet [7]	DeepTracking [6]	SAAConvLSTM [8]	TAAConvLSTM [8]	LOPR [9]	SCOPE	SCOPE++	SO-SCOPE
Inference Speed (FPS)	2.95	4.66	5.32	0.39	0.39	1.16	23.29	10.68	**34.75**
Model Size (MB)	12.44	37.17	**0.95**	21.64	22.02	1610.48	8.84	8.85	1.80
Memory Usage (GB)	0.70	0.71	**0.63**	1.23	1.23	5.00	0.66	0.66	0.66

that this is because the local environment map only provides useful information for static objects while the OSPA metric is biased towards dynamic objects (*e.g.,* pedestrians) since there tend to be more of them than static objects (*e.g.,* walls). Finally, the highest average OSPAs of SO-SCOPE-GTEMC also show the upper limit of the performance of our SO-SCOPE.

C. Failure Case Analysis

Although our SCOPE series demonstrates its accurate and robust future state prediction ability, there are still two typical failure cases. The first failure case is when the robot moves erratically or rotates rapidly and the robot motion compensation is inaccurate, as shown in Fig. 6a. We can easily see that when the robot suddenly rotates at the 6th prediction time step, the SO-SCOPE predictor of our default constant velocity motion model fails to predict the later future states. This is because our general constant velocity motion model fails to predict the robot's future motion, which is the key to OGM prediction. To support our analysis, we also provide prediction results for the SO-SCOPE-GTEMC (which shows an upper bound on the performance of SO-SCOPE with ground truth motions). From the last row of Fig. 6a, we can see that if we can accurately compensate for the robot's motion, our SO-SCOPE can still provide accurate future predictions after the 6th prediction timestep. Therefore, this failure issue can be easily alleviated by using more accurate robot motion models specific to the robots the researchers are using, as described in Sec. III-C1.

The second failure case is when the robot's field of view (FOV) is blocked by a large obstacle or objects such as pedestrians suddenly appear in the robot's FOV, as shown in Fig. 6b. We can see that when two pedestrians suddenly appear in the robot's FOV at the 6th prediction time step, our SO-SCOPE still misses the future states of these two pedestrians even with accurate motion compensation (*i.e.,* SO-SCOPE-GTEMC). This is because there is not enough information about the object's motion to feed into the neural network predictor, which is also important for OGM prediction. We believe that adding additional randomly sampled particles to the input OGMs or providing a longer OGM history can mitigate this occlusion or sudden appearance failure case. Exploring the OGM prediction problem in occluded or sudden appearance scenes will be our future work.

D. Computational Resource Utilization

While it is encouraging that our proposed SCOPE family of OGM predictors exhibits good prediction performance, we also want to evaluate the inference speed, model size, and memory usage of all tested OGM predictors. This is because mobile robots are resource-limited, and smaller model sizes, less memory usage, and faster inference speeds mean robots have a faster reaction time to face and handle dangerous situations in complex dynamic scenarios.

1) Baseline Comparison: Table II provides a comprehensive benchmark of resource usage, summarizing the inference speed, model size, and memory usage of nine predictors tested on a Jetson TX2 embedded computer equipped with a 256-core NVIDIA Pascal @ 1300MHz GPU and loaded with 8GB of memory. We can see that: 1) our SCOPE series achieves much faster inference than all other state-of-the-art baselines, especially our SO-SCOPE which can run at up to 35 FPS (about 89.1 times faster than the slowest SAAConvLSTM [8]and TAAConvLSTM [8]), 2) while our SCOPE and SCOPE++ models have the third-smallest model size, our proposed software-optimized SO-SCOPE model has the second-smallest model size and is approximately 894.7 times smaller than the largest LOPR [9] model, and 3) our SCOPE series has

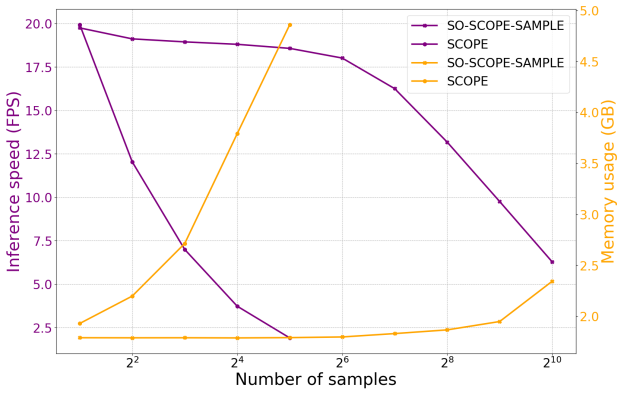


Fig. 7. Inference speed and memory usage of SCOPE and SO-SCOPE predictors with a different number of samples.

the second smallest memory usage, which is about 7.6 times smaller than LOPR which has the largest memory usage of 5GB. These show that our SCOPE family can have real-time inference speed, reasonable model size, and small memory usage, which is more hardware-friendly than other state-of-the-art OGM predictors and can be deployed on resource-constrained robots.

2) SCOPE Family Detailed Comparison: Then, we comprehensively compare the inference speed and memory usage of our software-optimized SO-SCOPE predictor with our SCOPE predictor (*i.e.,* a fast version of our SCOPE++) on a resource-constrained embedded computing device to show how our proposed software optimization approaches improve the embedded operation performance of SCOPE and SCOPE++. For a fair comparison with our SCOPE predictor, we set the SO-SCOPE predictor into a mode that generates samples where the prediction uncertainty statistics lookup table will provide the random variables. We compared their inference speed and memory usage when generating different numbers of samples, increasing as a power of 2 in the range $[2, 1024]$. Figure 7 shows their detailed comparison results.

As can be seen from Fig. 7, the inference speed of the SCOPE predictor drops significantly as the number of samples increases, with the lowest inference speed being about 2 FPS. In contrast, the inference speed of the SO-SCOPE hardly slows down as the number of samples increases until 128 samples are generated, and even with 1,024 samples it can still achieve around 6 FPS (*i.e.,* the speed bottleneck is generating a large number of samples rather than network inference). This demonstrates the significant advantage of using knowledge distillation techniques to compress our SCOPE++ network, since smaller networks lead to faster inference speeds, and using prediction uncertainty statistics lookup table to provide uncertainty estimates for our SO-SCOPE predictor, which speeds up the time-consuming sample generation process. Note that we could parallelize OGM sampling using our SO-SCOPE approach since it simply requires querying the lookup table, while SCOPE requires running the VAE model.

Figure 7 also shows that the memory usage of the SCOPE predictor increases significantly with the number of samples and can only generate up to 32 samples on a Jetson TX2 device due to its 8 GB memory limit. In contrast, the memory usage of the SO-SCOPE predictor barely increases with the number of samples until 1,024 samples are generated (*i.e.,* memory bottleneck is storing a large number of samples). It demonstrates the significant advantage of using the prediction uncertainty statistics lookup table to provide uncertainty estimates for our SO-SCOPE predictor in memory reduction.

E. Uncertainty Characterization

To characterize the uncertainty information of our SCOPE family of predictors, we use the Shannon entropy [75] to measure the uncertainty. For an OGM $\mathbf{o}$, the Shannon entropy is given by $H(\mathbf{o}) = -\sum_{c\in\mathbf{o}} p\log p + (1-p)\log(1-p)$, where the first equality comes from the assumption that cells c within an OGM are independent and the second equality comes from the definition of Shannon entropy for a Bernoulli distribution with parameter p (since each cell can either be occupied, with probability p, or free, with probability $1-p$). In our case, we have a predicted map, $\hat{\mathbf{o}}$ and data about the distribution of occupancy values $\tilde{c}$, which comes from the predicted occupancy value $\hat{c}$ along with the mixture model (6) and learned parameters ξ. Then, the entropy becomes

$$H(\hat{\mathbf{o}}) = \sum_{c\in\hat{\mathbf{o}}} E_{f_\xi(\hat{c})}[H(\tilde{c})], \tag{7a}$$

$$= -\sum_{c\in\hat{\mathbf{o}}} \int_0^1 [\tilde{c}\log\tilde{c} + (1-\tilde{c})\log(1-\tilde{c})]\, f_\xi(\tilde{c})d\tilde{c}, \tag{7b}$$

$$\approx -\sum_{c\in\hat{\mathbf{o}}} \frac{1}{M}\sum_{j=1}^{M} [\tilde{c}_j\log\tilde{c}_j + (1-\tilde{c}_j)\log(1-\tilde{c}_j)], \tag{7c}$$

where M is the number of predicted OGM samples and $\tilde{c}_j$ is the occupancy value of the cell in the jth predicted OGM $\tilde{\mathbf{o}}_j$.

Note that we can directly use (7b) to calculate the predicted OGM entropy of SCOPE, SCOPE++, and SO-SCOPE because we have the distribution $f_\xi(c)$ for OGM cell c from (6). We use the predicted mean map from OGM samples $\bar{\mathbf{o}} = \frac{1}{M}\sum_{j=1}^{M}\tilde{\mathbf{o}}$ instead of $\hat{\mathbf{o}}$ for SCOPE and SCOPE++, and the direct output of the SO-SCOPE network as $\hat{\mathbf{o}}$ for SO-SCOPE. On the other hand, we also use (7c) to estimate the predicted OGM entropy of SCOPE and SCOPE++, since we can obtain samples of predicted OGM through the VAE module. We also normalize the data by dividing the entropy by the total number of cells.

1) Experiment Setup: Our characterization of the uncertainty has three primary goals: 1) show that the generated OGMs are consistent with reality, 2) show that the SCOPE series converge to a consistent distribution, and 3) show that the uncertainty in the overall OGM increases with the number of objects in the scene. For the experiments, we use the OGM-Turtlebot data dataset and fix the prediction horizon to $\tau = 5$. We use the evaluation pipeline used to calculate OSPA Error on OGMs in [4] to count the number of objects in each OGM. Then, we randomly select 20 input sequences for each number of objects (from 1 to 12) and generate a total of 1,024 OGM samples for each input sequence using each SCOPE method.

2) Qualitative results: Our proposed SCOPE++, SCOPE, and SO-SCOPE have similar prediction performance and are able to generate prediction samples. Figure 8 showcases a diverse set of representative prediction samples. We see that

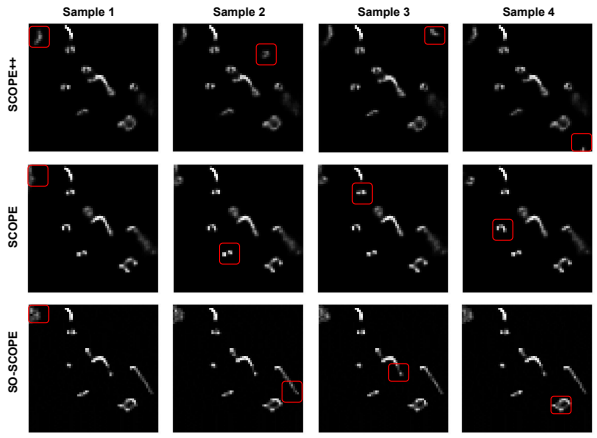
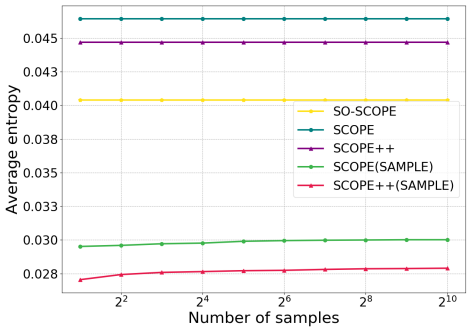
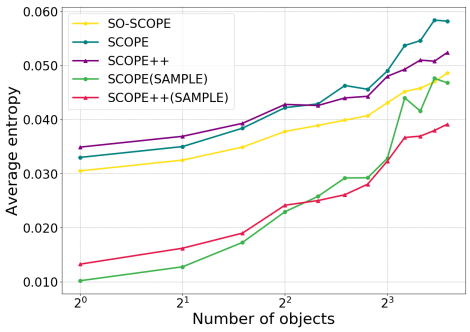


Fig. 8. A diverse set of prediction samples of our SCOPE++, SCOPE, and SO-SCOPE predictors tested on the OGM-Turtlebot2 dataset at the 5th prediction timestep. The red bounding boxes highlight the regions of variation in the predicted samples for the same input in each stochastic algorithm (each row), which demonstrates the diversity of our proposed stochastic predictors.



(a) Entropy vs. Number of samples



(b) Entropy vs. Number of objects

Fig. 9. Average entropy of our SCOPE, SCOPE++, and SO-SCOPE predictors at 5th prediction time step on OGM-Turtlebot2 dataset.

all three methods show variation across samples and generate realistic-looking structures, such as cylinders for legs and lines for walls. We also see that SCOPE++ shows more variation across samples than SO-SCOPE and SCOPE.

3) Quantitative results: One of the biggest differences between our proposed methods compared to state-of-the-art baselines is that the SCOPE family can provide uncertainty estimates. The goal of these tests is to analyze the output distribution of our SCOPE, SCOPE++, and SO-SCOPE predictors.

a) Convergence: First, we show how the entropy of the final probabilistic OGM changes as the number of OGM

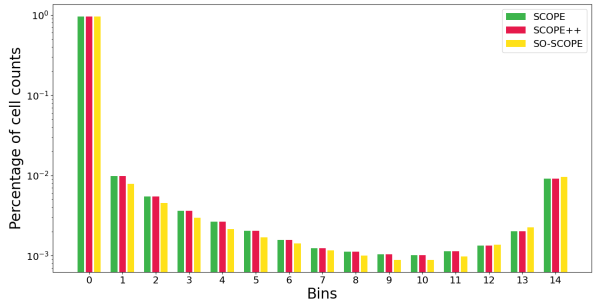


Fig. 10. Cell counts for each bin in the SCOPE family of predictors.

samples increases, with the hypothesis that it will level off at some value well below that of the entropy of a uniform distribution. Figure 9a shows that the average entropy of SCOPE(SAMPLE) and SCOPE++(SAMPLE) predictors (*i.e.,* computed via (7c)) starts to converge and plateau at a certain value after generating 128 samples, which is consistent with our hypothesis. Note that the entropy values for SCOPE, SCOPE++, and SO-SCOPE are flat lines because we use (7b) for calculation and no sampling is required. Furthermore, the differences among the three flat lines are small (*i.e.,* no more than 0.004), which indicates that our uncertainty quantification for SCOPE++ is correct and can provide reasonable uncertainty estimates for our SO-SCOPE predictor. The differences between them are mainly caused by the differences in their predicted OGMs. Figure 10 shows the differences in cell counts for each bin in their predicted OGMs. It is consistent with the entropy differences of our SCOPE, SCOPE++, and SO-SCOPE predictors, where the difference between SCOPE and SCOPE++ is smaller than the difference between SO-SCOPE and SCOPE++ (*i.e.,* SO-SCOPE has a smaller percentage of cell counts or lower entropy near bin 1 to bin 11).

However, we can see that there is a gap between the entropy calculated by (7b) using the mean map $\bar{\mathbf{o}}$ and the entropy calculated by (7c) (*i.e.,* SCOPE/SCOPE++ is higher than SCOPE(SAMPLE)/SCOPE++(SAMPLE)). We believe this gap stems from two factors. First, our assumption that cells c within an OGM are independent will overestimate the uncertainty values because actually knowing the occupancy of one cell should reduce the uncertainty of neighboring cells (and our learned latent representation in the VAE is encoding these correlations). Second, the mean map $\bar{\mathbf{o}}$ is different from individual samples $\tilde{\mathbf{o}}$, so the statistics differ slightly.

b) Scene Complexity: We also wish to show how the entropy of the final probabilistic OGM changes as the number of objects in the OGM increases, with the hypothesis that it will increase with the number of objects in it. Figure 9b clearly demonstrates that this is true, with the entropy of SCOPE/SCOPE(SAMPLE), SCOPE++/SCOPE++(SAMPLE) and SO-SCOPE predictors increasing with the number of objects. This confirms the intuition that there is more uncertainty in a scene with more (dynamic) objects in it.

VI. Uncertainty-Aware Navigation

We next test the applicability of our SCOPE series to practical mobile robot navigation in crowded dynamic scenes.

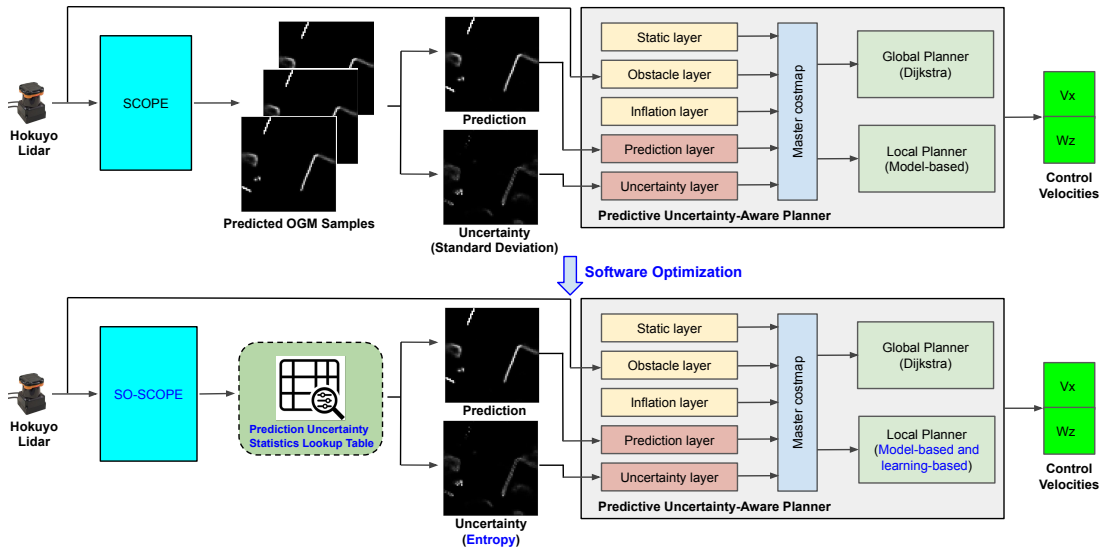


Fig. 11. System architectures of the SCOPE-based and SO-SCOPE-based predictive uncertainty-aware navigation planners. The blue font emphasizes the difference between the SCOPE-based navigation framework and the SO-SCOPE-based navigation framework. The basic process of our proposed navigation framework is as follows: first, the lidar data is also fed into our SCOPE or SO-SCOPE predictor to generate predicted OGM samples or lookup statistics from the prediction uncertainty statistics lookup table. Then, we can easily generate the prediction mean map and uncertainty map from these samples or the statistics. Finally, we create the prediction costmap layer and the uncertainty costmap layer, combine them into the master costmap, and obtain our SCOPE-based or SO-SCOPE-based predictive uncertainty-aware planners.

To leverage the uncertainty information in the environmental future states and provide robust and reliable navigation behavior, we propose a general predictive uncertainty-aware navigation framework (shown in Fig. 11) based on costmaps. Although this costmap-based framework loses the stochastic consistency of the entire navigation system, the biggest advantage is that it is used in the `move_base` ROS navigation framework and can be integrated with most currently existing control policies. Specifically, we use prediction and uncertainty costmaps to tell the robot the potentially dangerous areas ahead of it and how certain they are, which enables mobile robots to make safer nominal path plans, take proactive actions to avoid potential collisions, and improve navigation capabilities. To generate prediction and uncertainty costmaps from the output of our SCOPE family, we first binarize the predicted OGM and its uncertainty map using an occupancy threshold. Second, we initialize a constant cost value for each binarized prediction and uncertainty map grid cell and generate the initial prediction and uncertainty costmaps. Finally, we map each occupied grid cell of the prediction costmap and uncertainty costmap to a Gaussian obstacle value rather than a "lethal" obstacle value and obtain the final costmaps. This is because the predicted obstacles and uncertainty regions are not real obstacle spaces.

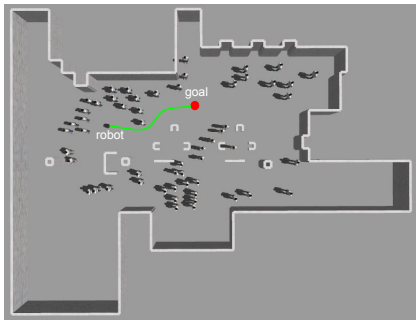
There are two versions: based on SCOPE and SO-SCOPE. The difference between SCOPE-based planner and SO-SCOPE-based planner is the method of generating prediction and uncertainty maps. The SCOPE-based planner needs to use the VAE module to generate prediction samples to provide a prediction map (*i.e.,* mean) and an uncertainty map (*i.e.,* standard deviation), which requires a large number of samples to provide accurate estimates and is time-consuming and memory-consuming. In contrast, our simple SO-SCOPE-based planner only needs to use a simple prediction uncertainty statistics lookup table to provide a prediction map (*i.e.,* SO-SCOPE output) and an uncertainty map (*i.e.,* entropy), which has fast running speed and small memory consumption. Note that the SCOPE-based navigation framework can only run with traditional model-based local planners (*e.g.,* DWA [10] and VO-based planners [76], [77]) in the resource-limited robots while the SO-SCOPE-based navigation framework can run with any types of model-based and learning-based local planners (*e.g.,* CNN [11], A1RC [12], and DRL-VO [13]).

A. Baselines and Evaluation Metrics

Since our proposed predictive uncertainty-aware navigation framework can be combined with different types of existing control policies, we define the following naming convention for predictive uncertainty-aware control strategy instances: `[policy]/[predictor]/[P/PU]`, where `[policy]` is the control policy used, `[predictor]` is the OGM predictor used, `P` means using only the prediction map, and `PU` means using both the prediction map and its uncertainty map.

Following this naming convention, we first use the model-based DWA [10] as the local planner to instantiate a SCOPE-based predictive uncertainty-aware planner (*i.e.,* DWA/SCOPE/PU) to demonstrate how prediction and its uncertainty information improve robot navigation performance. Then, we use the learning-based DRL-VO [13] as the local planner to instantiate a SO-SCOPE-based predictive uncertainty-aware planner (*i.e.,* DRL-VO/SO-SCOPE/PU) to demonstrate the SO-SCOPE predictor is hardware friendly and our SO-SCOPE-based navigation framework can be applied to any currently existing control policies.

We test these two predictive uncertainty-aware control policies, along with four state-of-the-art control policies: model-based DWA planner [10], supervised-learning-based CNN [11],



(a) Gazebo lobby



(b) Indoor hallway

Fig. 12. Robot navigation test environments in simulation and real-world. The Lobby Gazebo world has 4 crowd density configurations ranging from 15-45 (sampling interval 10) pedestrians.

DRL-based A1-RC [12], and DRL-based DRL-VO [13], and two ablation prediction-aware control policies without uncertainty maps: DWA/DeepTracking/P and DWA/SCOPE/P. Note that we use these four state-of-the-art baseline policies directly from their papers [10]–[13] without any retraining or parameter tuning. To evaluate the performance of navigation policies, we use the following four metrics from [11], [13]: success rate, average time, average length, and average speed.

B. Experiment Setup

Following the navigation evaluation settings in [11], [13], we conduct a total of 100 trials, and all control policies are tested by a Turtlebot2 robot with a maximum speed of 0.5 m/s, equipped with a Hokuyo UTM-30LX lidar and a ZED stereo camera, in the simulated Lobby Gazebo world with 15-45 pedestrians (sampling interval 10), as shown in Fig. 12a. The measurement range of Hokuyo lidar is set to $[0.1, 30]$ m, its FOV is $270°$, and its angular resolution is $0.25°$. The depth range of ZED camera is set to $[0.3, 20]$ m, and its FOV is $90°$.

In addition, we deploy our proposed DWA/SCOPE/PU and DRL-VO/SO-SCOPE/PU control policies on a real Turtlebot2 robot, which has the same configuration as the simulated robot but uses an NVIDIA Jetson AVG Xavier embedded computer as its main computing device. We conduct testing in an indoor hallway environment at Temple University during peak traffic periods between classes, the floor plan of which is shown in Fig. 12b. Note that considering the computational resources of Turtlebot2, we use the SCOPE predictor to only generate 8 predicted OGM samples at the 6th prediction time step (*i.e.,* 0.6 s) for the DWA/SCOPE/PU control policy and it cannot combine with any learning-based control policies. In contrast, our SO-SCOPE predictions do not suffer from such problems and can be combined with model-based or learning control strategies (*e.g.,* with DRL-VO in our instantiation).

C. Simulation Results

Table III summarizes these navigation results, where we observe three key phenomena. First, compared to DWA series variants (*i.e.,* DWA, DWA/DeepTracking/P, DWA/SCOPE/P, and DWA/SO-SCOPE/P), our proposed DWA/SCOPE/PU and DWA/SO-SCOPE/PU policies have the highest success rate in each crowd size, while DWA/SCOPE/PU has almost the shortest path length. This shows that the prediction costmap from our OGM predictors is able to help the currently existing model-based navigation policy (*i.e.,* DWA) to provide safer and shorter paths, and combining it with its associated uncertainty costmap can achieve a much better navigation performance (*i.e.,* PU is better than P alone). Furthermore, it shows that our software-optimized SO-SCOPE predictor can maintain good robot navigation performance even though it leads to longer paths compared to the SCOPE predictor. All these results demonstrate that our proposed SCOPE series predictors can improve safe robot navigation in crowded dynamic scenes.

The reasons why the OGM prediction and its uncertainty information can improve robot safe navigation can be explained qualitatively through Fig. 13. It shows the difference of nominal paths and costmaps generated by four planners (*i.e.,* DWA, DWA/SCOPE/P, DWA/SCOPE/PU, and DWA/SO-SCOPE/PU) in the simulated lobby environment. The default DWA planner [10] only cares about the current state of the environment and generates a costmap based on the perceived obstacles. The predictive DWA planner (*i.e.,* DWA/SCOPE/P) using the prediction map of our proposed SCOPE predictor can generate a costmap with predicted obstacles. The predictive uncertainty-aware DWA planner (*i.e.,* DWA/SCOPE/PU and DWA/SO-SCOPE/PU) using both the prediction map and uncertainty map of our proposed SCOPE predictor can generate a safer costmap with predicted obstacles and uncertainty regions. Note that the path generated by DWA/SO-SCOPE/PU is more tortuous than that of DWA/SCOPE/PU, which may explain why DWA/SO-SCOPE/PU requires longer paths to achieve a similar success rate as DWA/SCOPE/PU. These additional predicted obstacles and uncertainty regions of our proposed DWA/SCOPE/PU and DWA/SO-SCOPE/PU planners enable the robot to follow safer nominal paths and reduce collisions with obstacles, especially moving pedestrians. See the accompanying Multimedia for a detailed simulation navigation demonstration.

Second, our proposed DWA/SCOPE/P and DWA/SO-SCOPE/P have a higher success rate than DWA/DeepTracking/P at every crowd size, indicating that higher prediction accuracy can also improve safe navigation performance. Third, compared with all other state-of-the-art control policies, our proposed DRL-VO/SO-SCOPE/PU policy has the highest success rate and the fastest average speed in every situation. This suggests that our software-optimized SO-SCOPE predictor using a simple knowledge distillation network and prediction uncertainty statistical lookup table can also provide additional benefits for the learning-based safe navigation performance in crowded dynamic environments with varying crowd densities. Furthermore, it shows the potential capability of our software-optimized predictive uncertainty-aware navigation framework for different high-computational load learning-based policies.

TABLE III
NAVIGATION RESULTS AT DIFFERENT CROWD DENSITIES

Environment	Method	Success Rate	Average Time (s)	Average Length (m)	Average Speed (m/s)
Lobby world, 15 pedestrians	DWA [10]	0.94	12.21	5.09	0.42
	CNN [11]	-	-	-	-
	A1-RC [12]	0.94	14.36	6.40	0.45
	DRL-VO [13]	0.95	**11.34**	5.26	**0.46**
	DWA/DeepTracking/P	0.95	12.10	**5.03**	0.42
	DWA/SCOPE/P	0.93	12.62	5.06	0.40
	DWA/SCOPE/PU	**0.98**	12.79	5.06	0.40
	DWA/SO-SCOPE/P	0.94	12.12	5.10	0.42
	DWA/SO-SCOPE/PU	0.96	13.25	5.12	0.39
	DRL-VO/SO-SCOPE/PU	0.97	11.69	5.43	**0.46**
Lobby world, 25 pedestrians	DWA [10]	0.82	13.49	5.12	0.38
	CNN [11]	0.80	19.31	6.16	0.32
	A1-RC [12]	0.88	14.18	6.26	0.44
	DRL-VO [13]	0.92	**11.37**	5.29	**0.47**
	DWA/DeepTracking/P	0.87	12.67	**5.06**	0.40
	DWA/SCOPE/P	0.90	13.53	5.08	0.37
	DWA/SCOPE/PU	0.91	13.75	5.07	0.37
	DWA/SO-SCOPE/P	0.90	13.23	5.13	0.39
	DWA/SO-SCOPE/PU	0.92	14.26	5.19	0.36
	DRL-VO/SO-SCOPE/PU	**0.93**	11.95	5.52	0.46
Lobby world, 35 pedestrians	DWA [10]	0.82	14.18	5.15	0.36
	CNN [11]	0.81	14.30	5.40	0.38
	A1-RC [12]	0.77	16.81	6.89	0.41
	DRL-VO [13]	0.88	**11.42**	5.31	**0.46**
	DWA/DeepTracking/P	0.84	13.93	**5.10**	0.37
	DWA/SCOPE/P	0.86	13.79	5.12	0.34
	DWA/SCOPE/PU	0.89	14.90	5.12	0.34
	DWA/SO-SCOPE/P	0.86	14.03	5.16	0.37
	DWA/SO-SCOPE/PU	0.88	15.53	5.20	0.33
	DRL-VO/SO-SCOPE/PU	**0.92**	12.26	5.57	0.45
Lobby world, 45 pedestrians	DWA [10]	0.77	15.39	5.16	0.34
	CNN [11]	0.79	16.65	5.62	0.34
	A1-RC [12]	0.77	14.65	6.28	0.43
	DRL-VO [13]	0.81	**11.65**	5.37	**0.46**
	DWA/DeepTracking/P	0.78	15.23	**5.14**	0.34
	DWA/SCOPE/P	0.79	14.84	**5.14**	0.35
	DWA/SCOPE/PU	0.82	15.96	5.17	0.32
	DWA/SO-SCOPE/P	0.78	14.89	5.21	0.35
	DWA/SO-SCOPE/PU	0.80	16.29	5.27	0.32
	DRL-VO/SO-SCOPE/PU	**0.83**	12.24	5.56	0.45

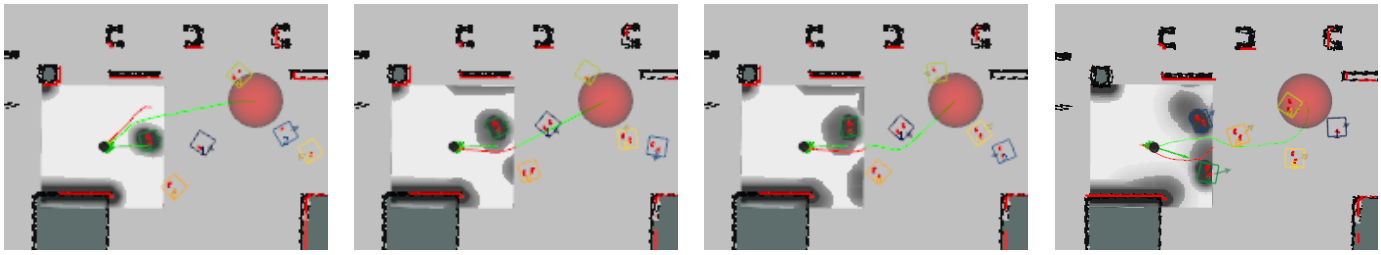
D. Hardware Results

Besides simulated experiments, we also conduct real-world experiments to demonstrate the applicability of our DWA/SCOPE/PU and DRL-VO/SO-SCOPE/PU policies. Both control policies use the `amcl` ROS package to provide the robot localization in known maps. Note that our DRL-VO/SO-SCOPE/PU control policy requires additional running of three deep learning networks (*i.e.,* YOLOv3 [78], SO-SCOPE, and DRL-VO [13]) and one optimization-based multiple hypothesis tracker (MHT) [79] to implement corresponding pedestrian detection, environment OGM prediction, navigation control, and pedestrian tracking. From the attached Multimedia and Fig. 14, we can see how our robot deployed with DWA/SCOPE/PU can actively avoid collisions with walking students crossing the hallway, safely avoid static students standing, and reach predefined goals by following predictive uncertainty-aware nominal paths, traveling a total length of 76.10 m and an average speed of 0.42 m/s. It demonstrates the real-world effectiveness of our proposed SCOPE predictor and DWA/SCOPE/PU planner.

In addition, from the attached Multimedia and Fig. 15, we can see that even with three learning-based blocks, our robot deployed with DRL-VO/SO-SCOPE/PU is still able to quickly and actively avoid collisions with walking students crossing the hallway and reach predefined goals by following predictive uncertainty-ware nominal paths, traveling a total length of 86.41 m and an average speed of 0.47 m/s. It demonstrates our software-optimized SO-SCOPE predictor is hardware friendly and our software-optimized predictive uncertainty-aware navigation framework can be combined with different high computational load learning-based algorithms.

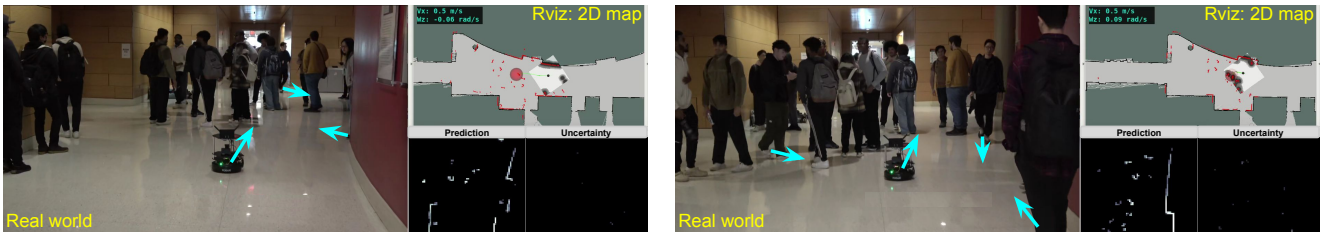
VII. CONCLUSION

In this article, we propose a family of hardware-friendly stochastic occupancy grid map prediction algorithms (*i.e.,* SCOPE++, SCOPE, and SO-SCOPE) that provides mobile robots with the ability to accurately and robustly predict the future states of complex, dynamic, human-occupied environments. Specifically, we first propose two VAE-based stochastic predictors (*i.e.,* SCOPE++ and SCOPE) that exploit information from robot motion, object motion, and static



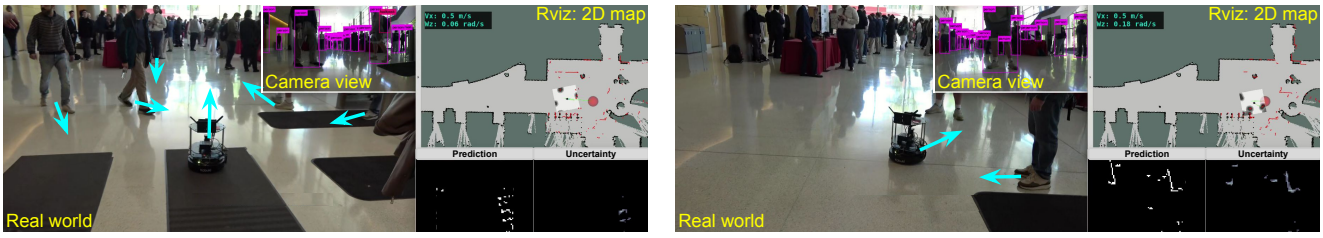
(a) DWA [10] (b) DWA/SCOPE/P (c) DWA/SCOPE/PU (d) DWA/SO-SCOPE/PU

Fig. 13. Robot reactions and their final costmaps generated by different DWA-based control policies in the simulated lobby environment, which shows that the final master costmap generated by the prediction map and its uncertainty map can provide safer path planning. The robot (black disk) avoids pedestrians (colorful square boxes, each color represents a pedestrian) and reaches the goal (red disk) according to the nominal path (green curve) and local path (red curve) planned by the costmap (square grey map).



(a) t (b) t+3

Fig. 14. Robot deployed with DWA/SCOPE/PU reactions to moving pedestrians in the indoor hallway with the high crowd density at different times.



(a) t (b) t+3

Fig. 15. Robot deployed with DRL-VO/SO-SCOPE/PU reactions to moving pedestrians in the indoor hallway with the high crowd density at different times.

scene geometry, and use a VAE-based network to fuse this useful information into the latent space and predict the distribution of future environment states. We then perform software optimization on them using knowledge distillation and uncertainty quantification operations to propose a hardware-friendly SO-SCOPE, which significantly improves the inference speed, addresses their sampling-based memory-intensive nature, and extends their practical applications in resource-constrained robots. Furthermore, we propose a novel predictive uncertainty-aware navigation framework by combining these proposed stochastic predictors with the costmap-based ROS navigation stack to improve the performance of current state-of-the-art model-based and learning-based control policies for safe navigation in crowded dynamic scenes.

We demonstrate that our proposed SCOPE family achieves smaller absolute error, higher structure similarity, and higher tracking accuracy than the other state-of-the-art image-based predictors on three different simulated and real-world datasets collected by three different robot models. It also provides a range of plausible and diverse future prediction states for complex stochastic environments. We further demonstrate through operational analysis and experiments on embedded computing devices that our proposed software-optimized SO-SCOPE prediction engine is much faster and more memory-efficient, allowing it to provide real-time inference with other resource-intensive algorithms in resource-constrained robots. Lastly, we demonstrate through simulated and hardware experiments that we can easily integrate the predicted maps into the existing navigation framework, all of which benefit from leveraging the prediction and uncertainty information of our SCOPE family. In summary, our proposed SCOPE predictors are hardware-friendly and improve mobile robots' ability to safely navigate through crowded dynamic environments.

ACKNOWLEDGMENT

This research includes calculations carried out on HPC resources supported in part by the National Science Foundation through major research instrumentation grant number 1625061 and by the US Army Research Laboratory under contract number W911NF-16-2-0189.

REFERENCES

[1] R. ED, "Types and applications of autonomous mobile robots," https://www.conveyco.com/blog/types-and-applications-of-amrs, July 2022, (Accessed on 08/20/2022).

[2] J.-u. Kim, "Keimyung hospital demonstrates smart autonomous mobile robot," https://www.koreabiomed.com/news/articleView.html?idxno=10585, Mar 2021, (Accessed on 08/20/2022).

[3] SICK, "Revolutionizing grocery shopping with mobile robots," https://sickusablog.com/revolutionizing-grocery-shopping-mobile-robots, Mar 2021, (Accessed on 08/20/2022).

[4] Z. Xie and P. Dames, "Stochastic occupancy grid map prediction in dynamic scenes," in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 06–09 Nov 2023, pp. 1686–1705. [Online]. Available: https://proceedings.mlr.press/v229/xie23a.html

[5] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong, and W.-c. Woo, "Convolutional LSTM network: A machine learning approach for precipitation nowcasting," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[6] P. Ondruska and I. Posner, "Deep tracking: Seeing beyond seeing using recurrent neural networks," in *Thirtieth AAAI conference on artificial intelligence*, 2016.

[7] V. L. Guen and N. Thome, "Disentangling physical dynamics from unknown factors for unsupervised video prediction," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 11 474–11 484.

[8] B. Lange, M. Itkina, and M. J. Kochenderfer, "Attention augmented convlstm for environment prediction," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 1346–1353.

[9] ——, "Lopr: Latent occupancy prediction using generative models," *arXiv preprint arXiv:2210.01249*, 2022.

[10] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.

[11] Z. Xie, P. Xin, and P. Dames, "Towards safe navigation through crowded dynamic environments," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Sep. 2021. [Online]. Available: https://doi.org/10.1109/IROS51168.2021.9636102

[12] R. Guldenring, M. Görner, N. Hendrich, N. J. Jacobsen, and J. Zhang, "Learning local planners for human-aware navigation in indoor environments," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 6053–6060.

[13] Z. Xie and P. Dames, "DRL-VO: Learning to navigate through crowded dynamic scenes using velocity obstacles," *IEEE Transactions on Robotics*, vol. 39, no. 4, pp. 2700–2719, Aug. 2023. [Online]. Available: https://doi.org/10.1109/TRO.2023.3257549

[14] ——, "Stochastic Occupancy Grid Map Prediction in Dynamic Scenes: Dataset," https://doi.org/10.5281/zenodo.7051560.

[15] A. Ess, K. Schindler, B. Leibe, and L. Van Gool, "Object detection and tracking for autonomous navigation in dynamic environments," *The International Journal of Robotics Research*, vol. 29, no. 14, pp. 1707–1725, 2010.

[16] D. Nuss, S. Reuter, M. Thom, T. Yuan, G. Krehl, M. Maile, A. Gern, and K. Dietmayer, "A random finite set approach for dynamic occupancy grid maps with real-time application," *The International Journal of Robotics Research*, vol. 37, no. 8, pp. 841–866, 2018.

[17] M. Itkina, K. Driggs-Campbell, and M. J. Kochenderfer, "Dynamic environment prediction in urban scenes using recurrent representation learning," in *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*. IEEE, 2019, pp. 2052–2059.

[18] M. Toyungyernsub, M. Itkina, R. Senanayake, and M. J. Kochenderfer, "Double-prong convlstm for spatiotemporal occupancy prediction in dynamic environments," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 13 931–13 937.

[19] M. Schreiber, S. Hoermann, and K. Dietmayer, "Long-term occupancy grid prediction using recurrent neural networks," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 9299–9305.

[20] M. Schreiber, V. Belagiannis, C. Gläser, and K. Dietmayer, "Motion estimation in occupancy grid maps in stationary settings using recurrent neural networks," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 8587–8593.

[21] W. Lotter, G. Kreiman, and D. Cox, "Deep predictive coding networks for video prediction and unsupervised learning," in *International Conference on Learning Representations*, 2016.

[22] M. Schreiber, V. Belagiannis, C. Gläser, and K. Dietmayer, "Dynamic occupancy grid mapping with recurrent neural networks," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 6717–6724.

[23] J. Dequaire, P. Ondrúška, D. Rao, D. Wang, and I. Posner, "Deep tracking in the wild: End-to-end tracking using recurrent neural networks," *The International Journal of Robotics Research*, vol. 37, no. 4-5, pp. 492–512, 2018.

[24] Y. Song, Y. Tian, G. Wang, and M. Li, "2d lidar map prediction via estimating motion flow with gru," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 6617–6623.

[25] H. Thomas, M. G. de Saint Aurin, J. Zhang, and T. D. Barfoot, "Learning spatiotemporal occupancy grid maps for lifelong navigation in dynamic scenes," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 484–490.

[26] H. Thomas, J. Zhang, and T. D. Barfoot, "The foreseeable future: Self-supervised learning to predict dynamic scenes for indoor navigation," *IEEE Transactions on Robotics*, 2023.

[27] N. Mohajerin and M. Rohani, "Multi-step prediction of occupancy grid maps with recurrent neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 10 600–10 608.

[28] Y. Cheng, D. Wang, P. Zhou, and T. Zhang, "Model compression and acceleration for deep neural networks: The principles, progress, and challenges," *IEEE Signal Processing Magazine*, vol. 35, no. 1, pp. 126–136, 2018.

[29] R. Mishra, H. P. Gupta, and T. Dutta, "A survey on deep neural network compression: Challenges, overview, and solutions," *arXiv preprint arXiv:2010.03954*, 2020.

[30] Y. He, X. Zhang, and J. Sun, "Channel pruning for accelerating very deep neural networks," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 1389–1397.

[31] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[32] E. L. Denton, W. Zaremba, J. Bruna, Y. LeCun, and R. Fergus, "Exploiting linear structure within convolutional networks for efficient evaluation," *Advances in neural information processing systems*, vol. 27, 2014.

[33] J.-H. Luo, J. Wu, and W. Lin, "Thinet: A filter level pruning method for deep neural network compression," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 5058–5066.

[34] D. Molchanov, A. Ashukha, and D. Vetrov, "Variational dropout sparsifies deep neural networks," in *International conference on machine learning*. PMLR, 2017, pp. 2498–2507.

[35] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "Scnn: An accelerator for compressed-sparse convolutional neural networks," *ACM SIGARCH computer architecture news*, vol. 45, no. 2, pp. 27–40, 2017.

[36] X. He, Z. Zhou, and L. Thiele, "Multi-task zipping via layer-wise neuron sharing," *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[37] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, "Mnasnet: Platform-aware neural architecture search for mobile," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 2820–2828.

[38] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," *arXiv preprint arXiv:1510.00149*, 2015.

[39] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks," *Advances in neural information processing systems*, vol. 29, 2016.

[40] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 2704–2713.

[41] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "Eie: Efficient inference engine on compressed deep neural network," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 243–254, 2016.

[42] P. Georgiev, S. Bhattacharya, N. D. Lane, and C. Mascolo, "Low-resource multi-task audio sensing for mobile and embedded devices via shared deep neural network representations," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 1, no. 3, pp. 1–19, 2017.

[43] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[44] S. I. Mirzadeh, M. Farajtabar, A. Li, N. Levine, A. Matsukawa, and H. Ghasemzadeh, "Improved knowledge distillation via teacher assistant," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 04, 2020, pp. 5191–5198.

[45] W. Son, J. Na, J. Choi, and W. Hwang, "Densely guided knowledge distillation using multiple teacher assistants," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 9395–9404.

[46] J. H. Cho and B. Hariharan, "On the efficacy of knowledge distillation," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 4794–4802.

[47] J. Yang, H. Zou, S. Cao, Z. Chen, and L. Xie, "Mobileda: Toward edge-domain adaptation," *IEEE Internet of Things Journal*, vol. 7, no. 8, pp. 6909–6918, 2020.

[48] A. J. Sathyamoorthy, J. Liang, U. Patel, T. Guan, R. Chandra, and D. Manocha, "Densecavoid: Real-time navigation in dense crowds using anticipatory behaviors," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 11 345–11 352.

[49] C. Chen, S. Hu, P. Nikdel, G. Mori, and M. Savva, "Relational graph learning for crowd navigation," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 10 007–10 013.

[50] D. Dugas, J. Nieto, R. Siegwart, and J. J. Chung, "Navrep: Unsupervised representations for reinforcement learning of robot navigation in dynamic human environments," *arXiv preprint arXiv:2012.04406*, 2020.

[51] K. Li, M. Shan, K. Narula, S. Worrall, and E. Nebot, "Socially aware crowd navigation with multimodal pedestrian trajectory prediction for autonomous vehicles," in *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2020, pp. 1–8.

[52] K. D. Katyal, G. D. Hager, and C.-M. Huang, "Intent-aware pedestrian prediction for adaptive crowd navigation," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 3277–3283.

[53] S. Liu, P. Chang, Z. Huang, N. Chakraborty, K. Hong, W. Liang, D. L. McPherson, J. Geng, and K. Driggs-Campbell, "Intention aware robot crowd navigation with attention-based interaction graph," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 12 015–12 021.

[54] G. Kahn, A. Villaflor, V. Pong, P. Abbeel, and S. Levine, "Uncertainty-aware reinforcement learning for collision avoidance," *arXiv preprint arXiv:1702.01182*, 2017.

[55] B. Lütjens, M. Everett, and J. P. How, "Safe reinforcement learning with model uncertainty estimates," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8662–8668.

[56] X. Tang, K. Yang, H. Wang, J. Wu, Y. Qin, W. Yu, and D. Cao, "Prediction-uncertainty-aware decision-making for autonomous vehicles," *IEEE Transactions on Intelligent Vehicles*, 2022.

[57] S. Sekiguchi, A. Yorozu, K. Kuno, M. Okada, Y. Watanabe, and M. Takahashi, "Uncertainty-aware non-linear model predictive control for human-following companion robot," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 8316–8322.

[58] G. Georgakis, B. Bucher, A. Arapin, K. Schmeckpeper, N. Matni, and K. Daniilidis, "Uncertainty-driven planner for exploration and navigation," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 11 295–11 302.

[59] S. M. LaValle and J. J. Kuffner Jr, "Randomized kinodynamic planning," *The international journal of robotics research*, vol. 20, no. 5, pp. 378–400, 2001.

[60] E. Wijmans, A. Kadian, A. Morcos, S. Lee, I. Essa, D. Parikh, M. Savva, and D. Batra, "Dd-ppo: Learning near-perfect pointgoal navigators from 2.5 billion frames," *arXiv preprint arXiv:1911.00357*, 2019.

[61] K. Katyal, K. Popek, C. Paxton, P. Burlina, and G. D. Hager, "Uncertainty-aware occupancy map prediction using generative networks for robot navigation," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 5453–5459.

[62] N. L. Baisa, "Derivation of a constant velocity motion model for visual tracking," *arXiv preprint arXiv:2005.00844*, 2020.

[63] C. Schöller, V. Aravantinos, F. Lay, and A. Knoll, "What the constant velocity model can teach us about pedestrian motion prediction," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1696–1703, 2020.

[64] T. Foote and M. Purvis, "Rep 103 – standard units of measure and coordinate conventions," https://ros.org/reps/rep-0103.html#axis-orientation, Dec 2014, (Accessed on 06/26/2024).

[65] S. Thrun, "Learning occupancy grid maps with forward sensor models," *Autonomous robots*, vol. 15, no. 2, pp. 111–127, 2003.

[66] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.

[67] S. Stanton, P. Izmailov, P. Kirichenko, A. A. Alemi, and A. G. Wilson, "Does knowledge distillation really work?" *Advances in Neural Information Processing Systems*, vol. 34, pp. 6906–6919, 2021.

[68] L. Beyer, X. Zhai, A. Royer, L. Markeeva, R. Anil, and A. Kolesnikov, "Knowledge distillation: A good teacher is patient and consistent," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 10 925–10 934.

[69] E. Beale, "Confidence regions in non-linear estimation," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 22, no. 1, pp. 41–76, 1960.

[70] H. Karnan, A. Nair, X. Xiao, G. Warnell, S. Pirk, A. Toshev, J. Hart, J. Biswas, and P. Stone, "Socially compliant navigation dataset (scand): A large-scale dataset of demonstrations for social navigation," *arXiv preprint arXiv:2203.15041*, 2022.

[71] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019, pp. 8026–8037.

[72] N. Ponomarenko, S. Krivenko, K. Egiazarian, V. Lukin, and J. Astola, "Weighted mean square error for estimation of visual quality of image denoising methods," in *CD ROM Proceedings of VPQM*, vol. 5. Scottsdale USA, 2010.

[73] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.

[74] D. Schuhmacher, B.-T. Vo, and B.-N. Vo, "A consistent metric for performance evaluation of multi-object filters," *IEEE Transactions on Signal Processing*, vol. 56, no. 8, pp. 3447–3457, 2008.

[75] C. E. Shannon, "A mathematical theory of communication," *ACM SIGMOBILE mobile computing and communications review*, vol. 5, no. 1, pp. 3–55, 2001.

[76] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *The International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, 1998.

[77] D. Wilkie, J. Van Den Berg, and D. Manocha, "Generalized velocity obstacles," in *2009 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2009, pp. 5573–5578.

[78] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[79] K. Yoon, Y.-m. Song, and M. Jeon, "Multiple hypothesis tracking algorithm for multi-target multi-camera tracking with disjoint views," *IET Image Processing*, vol. 12, no. 7, pp. 1175–1184, 2018.



Zhanteng Xie received the B.Eng. degree in electronic information engineering from Zhengzhou University, Zhengzhou, China, in 2015, the M.Eng. degree in information and communication engineering from the Harbin Institute of Technology, Harbin, China, in 2018, and the Ph.D. degree in mechanical engineering, Temple University, Philadelphia, USA. From 2018 to 2019, he was a research assistant in the Department of Electrical and Electronic Engineering at the Southern University of Science and Technology, Shenzhen, China.

His research interests lie at the intersection of robotics and machine learning, with a focus on environment perception, environment prediction, and autonomous robot navigation in crowded dynamic scenes.



Philip Dames received both his B.S. (summa cum laude) and M.S. degrees in mechanical engineering from Northwestern University, Evanston, IL, USA, in 2010 and his Ph.D. degree in mechanical engineering and applied mechanics from the University of Pennsylvania, Philadelphia, PA, USA, in 2015.

From 2015 to 2016, he was a Postdoctoral Researcher in Electrical and Systems Engineering at the University of Pennsylvania. Since 2016, he has been at Temple University, Philadelphia, PA, USA, where he is currently an Associate Professor of Mechanical Engineering and directs the Temple Robotics and Artificial Intelligence Lab (TRAIL). His research aims to improve robots' ability to operate in complex, real-world environments to address societal needs.

He is a member of IEEE and is the recipient of an NSF CAREER award.